

Categorical Message Passing Language (CaMPL): Syntax and Semantics

Robin Cockett* Daniel Kiyoshi Hashimoto[†] Alexanna Little Berg*
Priyaa Varshinee Srinivasan[‡]

September 11, 2026

Abstract

We introduce a novel functional-style concurrent programming language called Categorical Message Passing Language (CaMPL) which is designed using the mathematics of linear categories. This mathematical underpinning gives CaMPL programs useful properties such as deadlock freedom, and additionally, livelock freedom for programs without general recursive processes.

We explore CaMPL's type system through a series of code examples. The current proto-alpha version of the compiler and the abstract machine is implemented in Haskell. A reader is encouraged to experiment with writing CaMPL programs either using the online compiler <https://campl-app.vercel.app/> or by installing CaMPL from <https://campl-ucalgary.github.io> – our website has detailed instructions on how to run CaMPL code.

Contents

1	Introduction	2
2	A brief overview of CaMPL	4
3	Concurrent tier	5
3.1	Processes	5
3.2	Channels	6
3.2.1	Polarities	7
3.2.2	Built-in types	8
3.2.3	Custom types	12
3.2.4	Interacting with the outside world using service channels	15
3.3	Non-deterministic processes	16
3.4	Higher-order processes	19
4	Sequential tier	20

*University of Calgary

[†]Universidade Federal do Rio de Janeiro

[‡]Tallinn University of Technology, Estonia. This work was co-funded by the European Union and Estonian Research Council through the Mobilitas 3.0 (MOB3JD1227).

4.1	Functions	20
4.2	Data and codata types	20
4.3	Sequential control	22
5	Process commands manual	24
5.1	Connecting processes using <code>plug</code>	24
5.2	Calling processes	26
5.3	Equating channels using <code>id</code> and <code>neg</code>	26
5.4	Ending communication using <code>halt</code> and <code>close</code>	28
5.5	Complementary pairs	28
5.5.1	Sending and receiving messages using <code>put</code> and <code>get</code>	29
5.5.2	Channel activation using <code>hput</code> and <code>hcase</code>	30
5.5.3	Multi-process communication using <code>fork</code> and <code>split</code>	31
5.6	Controlled non-determinism using <code>race</code>	33
5.7	Service channel handles	34
6	Mathematical underpinning	34
6.1	Categorical semantics	35
6.2	Two-tiered logic	36
6.2.1	The logic of messages	36
6.2.2	The logic of message passing	36
7	Conclusion	37
A	Examples: Complete programs	42
A.1	Prelude	42
A.2	Higher order Hello World	43
A.3	Requesting user input continuously	44
A.4	Broadcasting a single message	45
A.5	Broadcasting an arbitrary number of messages	46
A.6	Non-deterministic server interacts with two clients	49
A.7	Pushing messages onto a Stack codata type	50
A.8	Memory cell	52

1 Introduction

Categorical Message Passing Language (CaMPL) is a functional-style concurrent programming language that uses asynchronous message-passing concurrency. It implements a type system based on linear logic that was described in Cockett and Pastro’s paper “The Logic of Message Passing” [CP09]. The primary result of their paper was defining and proving a concurrent analogue of the functional programming Curry-Howard-Lambek correspondence or proofs-as-programs principle [How80; Lam69; Lam72]. Their paper diverges from the approach taken by Abramsky et al. in [Abr93a; Abr93b; AM99; AGN00] which used exclusively the types and rules of linear logic that were introduced by Girard in [Gir87]. It also diverges from the popular approach of using linear logic to retrospectively develop a type system for the π -calculus (comprehensively

presented by Milner in [Mil93]) which has produced a significant body of work especially after the connection to session types was established [Hon93; BS94; KPT96; Bar+97; CP10; Wad12; TCP13; KMP19; AG19; QKB21]. Instead, Cockett and Pastro introduced a two-tiered logic that augments linear logic with new rules for passing messages of sequential data, such as strings or integers. Consequently, CaMPL is a two-tiered programming language. We motivate this design decision with the following excerpt from Cockett and Pastro’s paper:

We model message passing using a two tier logic. There is a logic for the messages whose proofs should be thought of as ordinary sequential programs. Then there is a logic of message passing which is built on top of the logic of messages. The two logics are really quite distinct: the message logic is concerned with what we classically view as computation, while the second logic is concerned with manipulating the channels of communication.

[...] However, even the briefest perusal of the rules, indicates that the logic concerned with managing channels is at least as complicated as the sequential programming logic. Furthermore, there are quite significant interactions between the two levels. [...] Thus, we believe programming language designers should be thinking in terms of developing integrated two tier languages in order to give high-level support for concurrent programming [...].

The categorical semantics Cockett and Pastro give for the two-tiered logic is a *linear actegory*. A linear actegory minimally consists of a monoidal category acting covariantly and contravariantly on a linearly distributive category, satisfying certain coherences [CP09; CS97b].

The concurrent type system provided by the message-passing logic ensures that processes can never be connected in a cycle. That is, at any instance in run-time, the topology of a program is guaranteed to be a finite acyclic graph consisting of processes as nodes and channels as edges. The benefit of this property is that problems caused by cycles, such as deadlocks and livelocks, do not occur [Lyb18]. However, similar to proving termination in the sequential case, livelock freedom is only guaranteed if we disallow general recursion. Deadlock freedom is still guaranteed, as is common in concurrent type systems based on linear logic [BS94; CP10; Wad12; TCP13; QKB21].

The first implementation of the CaMPL compiler was written by Kumar as described in [Kum18]. This version implemented sequential and concurrent type systems. It added features to define custom concurrent data types called protocols and coprotocols. The categorical semantics for protocols and coprotocols were given by Yeasin in [Yea12]. This compiler performed both a type inference and type checking. The second implementation of the compiler was written by Pon as described in [Pon21; Pon22]. This version re-implemented the existing features and added controlled non-determinism in the form of “races.” Changes to the categorical semantics to include non-determinism were outlined by Little in [Lit22; Lit23]. The most recent feature added to the compiler was higher-order message passing which allows processes to send messages that contain encoded processes which can be decoded and invoked when they are received. The implementation of this feature and the changes to the categorical semantics were written by Norouzbeigi in [Nor25; CN26]. The current implementation of CaMPL is available at <https://campl-ucalgary.github.io/>.

In this article, we illustrate CaMPL’s two-tiered type system and demonstrate the above features through a series of code examples. The brief overview in Section 2 considers two basic examples without explaining specific details. We discuss the concurrent tier of CaMPL, which corresponds to the message-passing logic, in Section 3. We discuss the sequential tier of CaMPL, which corresponds to the message logic, in Section 4. We will also discuss how the value of sequential data can be used to control a process’s execution. The key concepts of the above programming features will be covered by these two sections, and the corresponding channel types and process commands are summarized in Table 2. If one is interested in writing CaMPL programs, we provide finer details in Section 5. We also cover some more complicated examples. Finally, in Section 6, we briefly discuss the categorical and logical semantics described by Cockett and Pastro and explain how CaMPL emerged from them. In Appendix A, we provide examples of complete programs that were written by collecting code from the examples throughout this article.

2 A brief overview of CaMPL

As is tradition, we will introduce CaMPL with a “Hello World!” example program, shown in Example 1. We use *this style* whenever we introduce CaMPL terminology. We demonstrate a *process* named `helloworld` printing “Hello World!” by sending the string as a message on a *channel* named `console`. The console channel has type `Console` which is a special channel type built-in to the compiler to connect any CaMPL program to the terminal from which it is run.

Example 1: Hello World!

```

1 proc helloworld :: | Console => =
2   | console => -> do
3     hput ConsolePut on console
4     put "Hello World!" on console      -- sends message to the console
5     hput ConsoleClose on console
6     halt console                      -- closes console channel and halts

```

The `helloworld` process can be invoked by calling it in the main process, `run`, and giving it the console channel as follows:

```

7 proc run =
8   | console => -> helloworld( | console => ) -- creates helloworld process

```

In Example 2, we will demonstrate communication between two user-defined processes: `client` and `server`. The client will send a string to server, and server will receive it and echo it back. Then, client and server will halt. The main process will create the client and server processes, and it will use a *process command* called `plug` to connect them by a channel named `ch`.

Example 2: Server echos Client’s message

```

1 proc run =
2   | => -> plug                -- connects processes by shared channel ch
3     client( | => ch )          -- creates client process
4     server( | ch => )          -- creates server process

```

The CaMPL compiler will infer the *channel type* of `ch` using the process’ definitions.

The client process has an *output polarity channel* `ch` which connects it to the server. It sends its message with `put`, and it receives the server’s echo with `get`. Then, it uses `halt` to close the channel with the server and halt. We can define `client` as follows:

```
1 proc client =
2   | => ch ->           -- ch is an output polarity channel
3   on ch do
4     put "Hello Server!" -- sends message to the server
5     get echo           -- receives server's echo
6     halt
```

The server process has an *input polarity channel* `ch` which connects it to the client. It listens to the client and receives the client’s message with `get`, echos the message back with `put`, and finally closes the channel and halts with `halt`. We can define `server` as follows:

```
1 proc server =
2   | ch => ->           -- ch is an input polarity channel
3   on ch do
4     get msg           -- receives client's message
5     put msg           -- sends message back
6     halt
```

These simple programs conceptually exemplify how CaMPL programs are written. In fact, the lines of the code in the second program are out of order as the run process should be the final process defined in a program. We give a full program with the lines in order in Example 4. Furthermore, the runnable program obtained by reordering the lines still would not produce any observable effects since none of the processes are connected to the outside world.

A reader should be left with open questions such as “what was the type of channel `ch`?” “what is polarity?” and “what else can this language do?” We hope that these questions motivate the reader to continue on to the more complicated examples in the following sections.

3 Concurrent tier

The concurrent tier of CaMPL is concerned with managing interactions between processes along channels. At any instance in run-time, the topology of a program is a finite acyclic graph consisting of processes as nodes and channels as edges. The property of acyclicity is guaranteed by the type system, and it ensures that programs will never deadlock. Furthermore, a program without any general recursive processes will never deadlock or livelock.

The CaMPL compiler performs both a type inference and type check at compile-time to ensure the compiled program follows the type system. This section highlights the features of CaMPL that constitute its concurrent type system. We synthesize content from [Kum18; Pon21; Pon22; Nor25].

3.1 Processes

Processes are the main actors of a CaMPL program. A process is specified by the keyword `proc` followed by a process name, an optional type signature, and lists of its variables and channels.

In Example 1, the type signature of process `helloworld` was "`:: | Console =>`" which indicates that it has access to one channel of type `Console`. This channel is given the name `console` as indicated by "`= | console =>`" on the next line. We will discuss type signatures in Section 3.2.2.

The final process defined in a program, in which the program's execution begins, must have the name `run`. This process is the main process and the only process which can create *service channels*, such as the `Console`, for communication with the outside world. We will discuss service channels in more detail in Section 3.2.4.

For now, we will consider the definition of process broadcast in Example 3:

Example 3: Process that broadcasts a message from a single source

```

1 proc broadcast =
2   ack_msg | source => dest1, dest2 -> do
3     get msg on source           -- receive message from source
4     put msg on dest1           -- broadcast message to other processes
5     put msg on dest2
6     put ack_msg on source       -- send acknowledgement message to source
7     close source               -- close all channels and halt
8     close dest1
9     halt dest2 -- the last channel is closed with the halt command

```

Observe that line 2 consists of three comma-separated lists: 1) variables, e.g. `ack_msg`, 2) input polarity channels, e.g. `source`, and 3) output polarity channels, e.g. `dest1`, `dest2`. *Variables* are instances of sequential types and *channels* are instances of concurrent types. The channels left of `=>` are *input polarity* channels. The channels right of `=>` are *output polarity* channels.

Lines 3 - 9 constitute the *process body*. We say that the channels listed in line 2 are *in scope* of the process body which means that the process can perform operations, called *process commands*, on these channels. The process body may be a single process command, as shown in the `run` processes in Examples 1 and 2, or a `do` block of process commands, as shown in `broadcast`. Process commands performed on the same channel can be grouped together with "`on ch do`" as in Example 2 on line 3 of `client`. This allows one to omit a repetitive "`on ch`" after each command. Certain process commands may only be used as the last process command in a command block, for example an invocation of another process. We explain these finer details on process commands in Section 5.

Different process body definitions can be executed depending on the value of variables. We will discuss how sequential data can control a process's execution in Section 4.3.

Mutually recursive process definitions can be given using a `defn` statement and local process definitions can be given using a `where` statement inside a `defn` statement. Details of `defn` and `where` are given in Section 5.2.

We can depict processes using diagrams. We will use diagrams to illustrate some of the interactions processes can have along channels. A diagram of `broadcast` is shown in Figure 1. We use `-` to denote input polarity and `+` to denote output polarity.

3.2 Channels

A process *uses* process commands on a channel to interact with the process plugged into the other end. Two processes are connected by (at most) one typed channel that they use with opposite

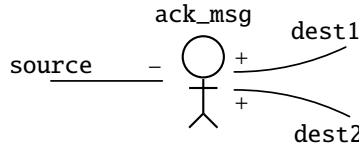


Figure 1: Diagram of process broadcast

polarities. The *type* of a channel defines the interaction that the processes will have over it, and its *polarities* define each process's role in the interaction. Thus, the process commands that a process can use on a channel depend on both type and polarity. The type of a channel can either be explicitly defined in the type signature of a process, or the compiler can infer the type of a channel based on how the processes on either end use it.

3.2.1 Polarities

A process is defined with two lists of the channels in its scope: input polarity channels and output polarity channels. A channel's polarity refers to which *end* of the channel the process uses. We say that a process using a channel with output polarity is "on the left end," and a process using a channel with input polarity is "on the right end." The input/output polarity terminology expresses a process-centric perspective, and the left/right end terminology expresses a channel-centric perspective.

Recall Example 2 in which `client` and `server` are connected by a channel `ch`. The `client` uses `ch` with output polarity (the left end) and `server` uses `ch` with input polarity (the right end).

Example 4: Server echos Client's message

```

1  proc client =
2    | => ch ->           -- channel ch has output polarity
3    on ch do
4      put "Hello Server!"
5      get echo
6      halt
7
8  proc server =
9    | ch => ->           -- channel ch has input polarity
10   on ch do
11     get msg
12     put msg
13     halt
14
15  proc run =
16    | => -> plug         -- program looks like client = ch = server
17    client( | => ch )     -- client is on the left
18    server( | ch => )     -- server is on the right

```

Recall that the type of a channel defines the interaction that the processes will have on it. In the first step of the above interaction, a message travels from left to right. Recall that the polarity of

the channel that each process uses defines that process's role in the interaction. The `client` uses `put` on its output polarity channel `ch` to send a message. Complementary to `put`, server uses `get` on its input polarity channel `ch` to receive the message. In the second step, the message travels in the opposite direction, so the type of `ch` is inferred as the dual of the previous step and the processes use the dual commands. Finer details on dual channel types and complementary pairs of process commands are given in Section 5.5.

Channel polarities allow one to define unambiguous interactions between processes. Without polarities to distinguish their roles, both `client` and `server` could use a `get` command in the first step of the interaction. Then, they would both wait for a message to be sent by the other, thereby causing a deadlock. For example, suppose we do not consider polarity and rewrite our code as follows:

Example 5: Code without channel polarity (this code will not compile)

```

1  proc client =
2    | ch ->
3    on ch do
4      get msg           -- receives server's message
5      put "Hello Server!" -- sends message to the server
6      halt
7
8  proc server =
9    | ch ->
10   on ch do
11     get msg           -- receives client's message
12     put "Hello Client!" -- sends message to client
13     halt
14
15  proc run =           -- if we could compile this program, it would deadlock!
16    | => -> plug
17    client( | ch )     -- client process has channel ch (no defined polarity)
18    server( | ch )     -- server process has channel ch (no defined polarity)

```

Neither process will be able to continue past their `get` command because nothing is actually `put` on the channel first. This is precisely what a deadlock is. In this simple example, it is easy to see the mistake, but in more complicated programs, the type system with polarities has the ability to catch these sorts of errors. Each built-in channel type in CaMPL specifies the complementary roles of the processes on each end of the channel to ensure it is unambiguous.

3.2.2 Built-in types

Built-in channel types define the fundamental interactions that are permitted to occur between processes. The type of a channel can either be explicitly defined in the type signature of a process, or the compiler can infer the type of a channel based on how the processes on either end use it.

As mentioned in Section 3.1, process definitions may include a type signature with three comma-separated lists of sequential types, input concurrent types, and output concurrent types. Variables and channels are bound to types in the order they appear.

We modify Example 2 and include the processes' type signatures. To effectively illustrate the types, we redefine server to have a `server_id` variable that it will send back to client:

Example 6: Server replies to Client with server id

```

1  proc client :: | => Put([Char]|Get(Int|TopBot)) =      -- type signature
2    | => ch ->
3      on ch do
4        put "Hello Server!"
5        get server_id
6        halt
7
8  proc server :: Int | Put([Char]|Get(Int|TopBot)) => =  -- type signature
9    server_id | ch => ->      -- new server_id variable
10   on ch do
11     get msg
12     put server_id          -- sends server_id back
13     halt
14
15  proc run :: | => =      -- type signature
16    | => -> plug
17      client( | => ch )
18      server( 1234 | ch => )  -- server process created with server_id

```

Observe that the type signature of `run` does not contain any types because it is not using any variables or any channels to interact with other processes.

Notice that `ch` has type `Put([Char]|Get(Int|TopBot))` which is constructed inductively. The outer-most layer `Put([Char]|...)` defines the first step of the interaction in which a string `msg` travels from left to right (client to server). Accordingly, client uses `put` on line 4 to send, and server uses `get` on line 11 to receive. In the next layer `Get(Int|...)`, an integer `server_id` travels from right to left (server to client). Accordingly, client uses `get` on line 5 to receive, and server uses `put` on line 12 to send. Finer details about these commands are given in Section 5.5.1. The inner-most layer `TopBot` is the base case and final step. We can `close` the channel or, as above, `halt` by closing the final open channel. Finer details about these commands are given in Section 5.4.

From Example 6, we can see that a channel of type `Put` indicates messages will travel from left to right, type `Get` indicates messages will travel from right to left, and type `TopBot` indicates the interaction is over and the channel will be closed. The diagrams in Figures 2 and 3 visualize the direction messages travel on each channel type.

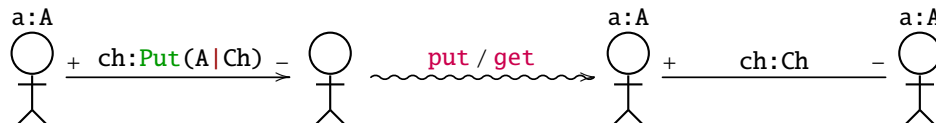


Figure 2: Message passing on `Put` channels

Next, in Example 7, we will consider a compound channel type that allows bundling of two (or more) channels into one. The types `(*)`, called *tensor*, and `(+)`, called *par*, come from multiplicative



Figure 3: Message passing on **Get** channels

linear logic and allow changes to be made to the network of processes while ensuring no cycles are introduced. By unbundling the channels, two new channels are created and passed into two new processes.

We again modify Example 2 to demonstrate (*). We define a new process `two_clients` that uses **fork** on line 9 to unbundle its output channel `two_ch` and create two instances of `client` that both send a message to a modified server process. This server uses **split** on line 15 to unbundle its input channel `two_ch` into two channels, `ch1` and `ch2`. After unbundling, `two_ch` is no longer in scope in either process.

Example 7: Client forks to create two new processes

```

1  proc client :: | => Put([Char]|TopBot) =
2    | => ch ->
3      on ch do
4        put "Hello Server!"      -- client sends string
5        halt
6
7  proc two_clients :: | => Put([Char]|TopBot) (*) Put([Char]|TopBot) =
8    | => two_ch ->
9      fork two_ch as             -- unbundles channels ch1 and ch2
10     ch1 -> client( | => ch1)    -- creates two new client processes
11     ch2 -> client( | => ch2)    -- that each get one channel
12
13  proc server :: | Put([Char]|TopBot) (*) Put([Char]|TopBot) => =
14    | two_ch => -> do
15      split two_ch into ch1, ch2 -- unbundles channels ch1 and ch2
16      on ch1 do                  -- interacts with first client
17        get msg
18        close
19      on ch2 do                  -- interacts with second client
20        get msg
21        halt
22
23  proc run :: | => =
24    | => -> plug
25    two_clients( | => two_ch )    -- creates a single two_clients process
26    server( | two_ch => )

```

Observe that `two_ch` has type `Put([Char]|TopBot)(*) Put([Char]|TopBot)`. The outer-most (*) type indicates that the process on the left (`two_clients`) will be replaced by two new processes that are both connected to the process on the right (`server`). The next layers indicate how the interactions

between the processes on the two new channels will proceed.

What if we wanted the process on the right to be replaced by two new processes instead? Then, we would use (+) which is dual to (*). The diagrams in Figures 4 and 5 visualize the change in the process network from each channel type. An example demonstrating a channel with type (+) and finer details about the corresponding process commands are given in Section 5.5.3.

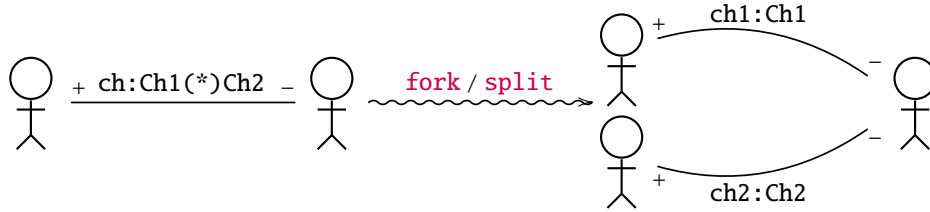


Figure 4: Change in process network from (*) channels

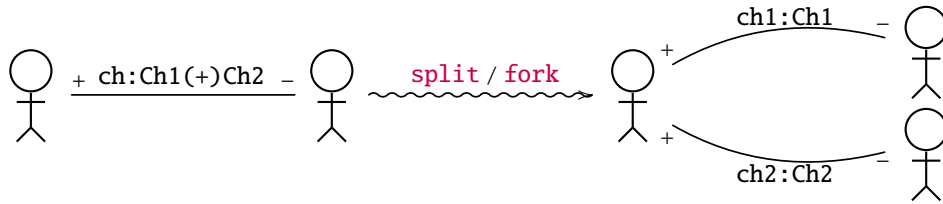


Figure 5: Change in process network from (+) channels

Lastly, the Neg channel type negates a given channel type. This means that the direction of the interaction is reversed, so it is in the same direction as the *dual* type. A Neg(Put) channel can be used like a Get channel (and vice versa), and a Neg(*) channel can be used like a (+) channel (and vice versa).

Recall process broadcast from Example 3. The type signature of broadcast is as follows:

```
1 proc broadcast :: [Char] | Put([Char] | Get([Char] | TopBot))
  => Put([Char] | TopBot), Put([Char] | TopBot) =
2   ack_msg | source => dest1, dest2 ->
```

Note that the type of dest1 is Put([Char] | TopBot). We will refer to the destination process on the other end of dest1 as proc1. Notice that proc1 must use dest1 with input polarity.

Suppose we actually want proc1 to use dest1 with output polarity. Instead of changing broadcast to connect with proc1 directly, we can write another process, send_to_dest1, that connects to proc1. Our run process would be defined as follows:

```
1 proc run =
2   | => -> plug
3   sender( | => source)
4   broadcast("message broadcasted" | source => dest1, dest2)
5   send_to_dest1( | dest1, neg_dest1 => ) -- type of dest1 is Put
6   proc1( | => neg_dest1) -- type of neg_dest1 is Neg(Put)
7   proc2( | dest2 =>)
```

Messages will travel along `dest1` from left to right and then in the opposite direction on `neg_dest1`. Since `dest1` is type `Put`, `neg_dest1` can be `Neg(Put)`. We define process `send_to_dest1` as follows:

Example 8: Using a `Neg(Put)` channel instead of a `Get` channel

```

1 proc send_to_dest1 :: | Put([Char] | TopBot), Neg(Put([Char] | TopBot)) => =
2   | source, neg_dest1 => -> do    -- send a message on neg_dest1 like a Get type
3     on source do                -- receive message from source via broadcast
4       get msg
5       close
6     plug
7       neg_dest1, dest1 => ->      -- create a channel dest1
8       neg_dest1 |=| neg dest1    -- negate it and identify with neg_dest1
9       => dest1 ->                -- use dest1 to forward message to proc1
10      on dest1 do
11        put msg
12        halt

```

To connect to this process, `proc1` would also need to be defined to have a `Neg(Put)` channel instead of a `Get` channel because they are not actually the *same* type. Additionally, `proc1` would need to include code similar to lines 6 - 12 to be able to use the `Neg(Put)` channel. A complete program that implements `proc1` is given in Appendix A.4. Another example demonstrating a `Neg` channel and finer details on `neg` and identification `|=|` are given in Section 5.3.

Table 1 summarizes the interactions provided by the built-in channel types. We denote an argument for a sequential type with `A` and a concurrent channel type with `Ch`, `Ch1`, and `Ch2`. Channel types that take another channel type as an argument are recursive cases. The outer-most layer is the first interaction that will take place, and after that, the interaction will continue according to the type of the channel given as the argument.

Channel type	Description of interaction
<code>TopBot</code>	End communication on the channel (this is the base case).
<code>Put(A Ch)</code>	A message of type <code>A</code> travels from left to right (output to input polarity).
<code>Get(A Ch)</code>	A message of type <code>A</code> travels from right to left (input to output polarity).
<code>Ch1(*)Ch2</code>	Left process becomes two processes both connected to right process.
<code>Ch1(+)Ch2</code>	Right process becomes two processes both connected to left process.
<code>Neg(Ch)</code>	Direction of interaction for <code>Ch</code> type is reversed.

Table 1: Summary of built-in channel type interactions.

3.2.3 Custom types

We have covered the basic channel types, but the processes we have written so far cannot send variable numbers of messages. Recall process `broadcast` from Example 3. This process was only able to broadcast a single message from the source. To allow `broadcast` to call itself recursively until all messages are sent, the channel type needs to be able to change from `Put` to `TopBot`. This can be

achieved by defining custom channel types, called *protocols* and *coprotocols*, which can be recursive. Protocols and coprotocols are similar to session types as one process chooses an interaction from a pre-defined set of possible interactions and the other process defines its behaviour for each case. Non-recursive protocols and coprotocols come from additive linear logic. Recursive protocols and coprotocols are the concurrent analogue of inductive and coinductive data types in functional programming languages. However, a coprotocol is simply a dual protocol, which is not true for sequential inductive and coinductive data types.

Consider the protocol `SendMsgs`:

Example 9: Protocol for sending an arbitrary number of messages

```
1 protocol SendMsgs(A | ) => S =           -- sends messages of type A
2   SendMsg :: Put(A | S) => S             -- handle to send another message
3   CloseCh :: TopBot => S                 -- handle to finish sending messages
```

This protocol allows an arbitrary number of messages of type `A` to be passed on a `Put` channel. When we use it as a channel type, we will instantiate the sequential type variable `A`. The *handles* represent the set of valid interactions, or session types, that may take place on the channel. The handle `SendMsg` sets the channel type to `Put` to allow another message to be sent. Once all messages are sent, `CloseCh` can be used to change the channel type to `TopBot`.

If we want broadcast to also send an acknowledgement message on source, as it did originally, we need to define another protocol with a handle that sets the channel to send a message and receive a string in response. For example:

Example 10: Protocol for sending messages and receiving acknowledgement messages

```
1 protocol SendMsgsWithAck(A | ) => S =
2   SendMsgWithAck :: Put(A | Get([Char] | S)) => S -- handles have new names
3   CloseAckCh :: TopBot => S
```

We need to use different handle names than we did in `SendMsgs` because all handle names must be globally unique. Neither protocol has concurrent type variables, but a concurrent type variable `X` would be given as `SendMsgsWithAck(A | X)`. The name of a protocol, its handles, and its sequential and concurrent type variables are required to begin with an uppercase alphabet character.

We can modify broadcast to use the `SendMsgs` and `SendMsgsWithAck` protocols as follows:

Example 11: Process that broadcasts messages from a single source

```
1 proc broadcast :: [Char] | SendMsgsWithAck([Char] | ) => SendMsgs([Char] | ),
   SendMsgs([Char] | ) =
2   ack_msg | source => dest1, dest2 -> do
3     hcase source of           -- check whether there is another message
4       SendMsgWithAck -> do
5         get msg on source      -- receive message
6         on dest1 do           -- broadcast message to other processes
7           hput SendMsg        -- indicate there is another message
8           put msg              -- send that message
9         on dest2 do
10          hput SendMsg
11          put msg
```

```

12     on source do          -- send acknowledgement to source
13         put ack_msg
14         broadcast(ack_msg | source => dest1, dest2) -- recurse
15     CloseAckCh -> do      -- close all channels and halt
16         close source
17     on dest1 do
18         hput CloseCh      -- indicate source is finished
19         close             -- close channel
20     on dest2 do
21         hput CloseCh
22         halt

```

The sequential type variable `A` is instantiated with `[Char]` on line 1 to indicate we are broadcasting strings. A handle is received on the input polarity channel `source` using `hcase` on line 3. Process bodies for each handle are defined on lines 4 - 14 and 15 - 22. On lines 7 and 18, a communication session is initiated on the output polarity channel `dest1` by *activating* it with a handle using `hput`. This sets the channel type as specified by the handle. Finer details on these process commands are given in Section 5.5.2.

The only difference between protocols and coprotocols is the direction that handles travel on the channel. Protocol handles travel in the same direction as messages on a `Put` channel. For example, `broadcast` receives handles on `source` and sends handles on its destination channels. Coprotocol handles travel in the same direction as messages on a `Get` channel.

A `send_to_dest1` process that forwards an arbitrary number of messages to `proc1` could use a `Neg(SendMsgs([Char] | ))` channel. Alternatively, we could use a coprotocol version of `SendMsgs` instead. We can define a coprotocol `CoSendMsgs` as follows:

Example 12: Coprotocol for sending an arbitrary number of messages

```

1 coprotocol S => CoSendMsgs(A | ) =
2   CoSendMsg :: S => Get(A | S)
3   CoCloseCh :: S => TopBot

```

Notice that the state variable `S` is on the left side of `=>` instead of the right side like in the `SendMsgs` and `SendMsgsWithAck` definitions. We rewrite `send_to_dest1` using `CoSendMsgs`:

Example 13: Using a `CoSendMsgs` channel instead of a `Neg(SendMsgs)` channel

```

1 proc send_to_dest1 :: | SendMsgs([Char] | ), CoSendMsgs([Char] | ) => =
2   | source, dest1 => ->
3   hcase source of          -- check whether there is another message
4     SendMsg -> do
5       get msg on source
6       on dest1 do
7         hput CoSendMsg      -- use coprotocol handle
8         put msg
9       send_to_dest1( | source, dest1 => ) -- recurse
10  CloseCh -> do
11    close source
12    on dest1 do

```

```

13      hput CoCloseCh
14      halt

```

From this example, we can see that a coprotocol streamlines the process of writing code for a negated protocol rather than being a unique feature. A complete program based on this example is given in Appendix A.5.

Another use case for protocols and coprotocols is the creation of infinitely bundled channel types using (*) or (+). This is demonstrated in Example 26 and briefly discussed in Example 31.

3.2.4 Interacting with the outside world using service channels

The examples we have considered so far, with the exception of Example 1, will not produce any observable effects to a user since they have not been connected to the outside world. Recall and observe that Example 1 had the only run process that was defined with a non-empty list of channels in its scope. *Service channels* are the channels that a run process can be defined with. The service channel types include *Console*, *Terminal*, and *Timer*. They are implemented as special built-in protocols and coprotocols on which processes must always use *hput*. The compiler has built-in service processes which implement the corresponding *hcase* to provide the service.

A *Console* allows a user to interact through the terminal in which the program was run. A *Terminal* opens a new Alacrity¹ terminal for user interaction. A *Timer* can be used to implement time-out features using controlled non-determinism, which we will discuss in Section 3.3. A complete list of the handles for these types is included in Section 5.7. For now, we will focus on the following handles:

Example 14: Service channels

```

1 protocol Terminal => S =
2   StringTerminalPut :: Put([Char]|S) => S
3   StringTerminalGet :: Get([Char]|S) => S
4   StringTerminalClose :: TopBot => S
5
6 coprotocol S => Console =
7   ConsolePut :: S => Get([Char]|S)
8   ConsoleGet :: S => Put([Char]|S)
9   ConsoleClose :: S => TopBot

```

A Prelude with definitions of the service channel protocols and coprotocols and some simple but useful functions is given in Appendix A.1. This can be copied to a Prelude.mpl file that one can include and use. Consider a modification of Example 2 in which we include Prelude to give client a *Terminal* to receive user input and to give server a *Console* to log the messages it receives:

Example 15: Client and server using service channels

```

1 include Prelude
2
3 proc client :: | => SendMsgs([Char]| ), Terminal =
4   | => ch, terminal -> do

```

¹<https://alacrity.org>


```

5      on terminal do
6          hput StringTerminalPut
7          put "Hello User! Please enter message in terminal:"
8          hput StringTerminalGet
9          get msg                                -- receive user input
10     on ch do
11         hput SendMsg
12         put msg                                -- forward to server
13         hput CloseCh
14         close
15     on terminal do                            -- confirm then halt when user is ready
16         hput StringTerminalPut
17         put "Sent message to server. Press ENTER to close terminal."
18         hput StringTerminalGet
19         get _                                  -- wait for user before closing terminal
20         hput StringTerminalClose
21         halt
22
23 proc server :: | SendMsgs([Char]| ), Console => =
24     | ch, console => -> do
25         hcase ch of
26             SendMsg -> do                    -- log each message received
27                 get msg on ch
28                 on console do
29                     hput ConsolePut
30                     put "message from user: " ++ msg -- append msg and print
31                     server( | ch, console => )
32             CloseCh -> do                    -- halt after all messages received
33                 close ch
34                 on console do
35                     hput ConsoleClose
36                     halt
37
38 proc run :: | Console => Terminal =
39     | console => terminal -> plug
40     client( | => ch, terminal )
41     server( | ch, console => )

```

Note that on line 30 we use an infix concatenation function from the Prelude. This function will be defined in Example 20. Additionally, a `SendMsgs` definition must be added to run this program.

3.3 Non-deterministic processes

Recall Example 7 in which two client processes each sent a message to server. Notice that server received the messages in a pre-determined order.

Consider the case when server echos these messages back. With the features we have used so far, server must have the interactions in a pre-determined order. It would wait for the first client before starting its interaction with the second client, so the second client would also wait for the

first client. This is parallel but not concurrent! For true concurrency, server should dynamically opt to have interactions in the order it receives messages. We call this *controlled non-determinism* based on the outcome of a *race* between the channels. A server that calls a locally defined process `non_deterministic_server`, which holds a race using the `race` command, is shown in Example 16:

Example 16: Server echos each client in the order it receives messages

```

1 defn
2   proc server =      -- echos msgs without making clients wait for each other
3   | two_ch => -> do
4     split two_ch into ch1, ch2
5     server_non_deterministic( | ch1, ch2 => )
6
7 where defn          -- locally defined processes given after where
8   proc server_non_deterministic :: |
9     Put([Char]|Get([Char]|TopBot)), Put([Char]|Get([Char]|TopBot)) => =
10  | ch1, ch2 => -> do
11    race              -- controlled non-determinism via a race
12    ch1 -> server_deterministic( | ch1, ch2 => )      -- client on ch1 wins
13    ch2 -> server_deterministic( | ch2, ch1 => )      -- client on ch2 wins
14
15  proc server_deterministic :: |
16    Put([Char]|Get([Char]|TopBot)), Put([Char]|Get([Char]|TopBot)) => =
17  | winner, loser => -> do
18    on winner do      -- interacts with client it received a message from first
19      get msg
20      put msg
21      close
22    on loser do       -- interacts with other client
23      get msg
24      put msg
25      halt

```

Races can be held for any number of channels on which a process is waiting for a message. The `race` on line 10 consists of channels `ch1` and `ch2` as indicated by lines 11 and 12, respectively. A process body is defined for each channel in the race, and the process will execute according to the winning channel's corresponding process body. If `ch1` wins, line 11 will execute and `server_deterministic` will be called with `ch1` in the argument for the winner channel. If `ch2` wins, line 12 will execute and `server_deterministic` will be called with `ch2` in the argument for the winner channel.

Notice that the type signatures of `server_deterministic` and `server_non_deterministic` are the same. This is because races do not change channel types; they only change the order in which the steps of the interactions take place. Finer details on the `race` command are given in Section 5.6. We discuss locally defined processes in Section 5.2. A complete program based on this example is given in Appendix A.6.

Another use case for races is a time out feature using the `Timer` service channel:

```

1 coprotocol S => Timer =
2   Timer :: S => Get(Int|S (*) Put(())|TopBot))
3   TimerClose :: S => TopBot

```

This service is a coprotocol, so we use it on an input polarity channel. Thus, the `Get(Int|...)` type of the `Timer` handle allows one to `put` an integer with a time limit in microseconds. To set a timer for 60 seconds, that is 60 000 000. The `(*)` type allows one to split the channel and reuse the recursive part to set arbitrarily many timers. Finally, the `Put(())|...` type in the non-recursive part can be used in a `race`. We modify Example 15 to demonstrate this:

Example 17: Client that times out the Terminal after 60 seconds.

```

1  proc client :: | Timer => SendMsgs([Char]| ), Terminal =
2    | timer => ch, terminal -> do
3      on terminal do
4        hput StringTerminalPut
5        put "Hello User! Please enter message. Terminal will time out in 60 s."
6        hput StringTerminalGet
7      on timer do
8        hput Timer
9        put 600000000                                -- set time limit for 60 seconds
10     split timer into new_timer, times_up
11     on new_timer do                                  -- no recursion, so close new_timer
12       hput TimerClose
13       close
14     race
15       times_up -> do                                -- if the timer wins, user timed out
16         on times_up do
17           get ()
18           close
19         on ch do                                     -- don't forward anything to server
20           hput CloseCh
21           close
22         on terminal do                               -- halt after msg is finally received
23           get msg
24           hput StringTerminalPut
25           put "User timed out. Message not sent."
26           hput StringTerminalClose
27           halt
28       terminal -> do                                  -- if user wins, run original code
29         get msg on terminal
30         on ch do
31           hput SendMsg
32           put msg
33           hput CloseCh
34           close
35         on terminal do
36           hput StringTerminalPut
37           put "Sent message to server. Press ENTER to close terminal."
38           hput StringTerminalGet
39           get _
40           hput StringTerminalClose

```

```

41         close
42     on times_up do                                -- then close the timer and halt
43         get ()
44         halt

```

The current implementation of the `Timer` service does not allow one to cancel a timer. Thus, even if a user provides input within the time limit, the above process cannot halt until the time limit has elapsed and it receives a unit on line 43.

3.4 Higher-order processes

In sequential functional programming, higher-order functions take other functions as input or produce functions as output. CaMPL supports a concurrent analogue of this concept. *Higher-order processes* encode other processes as sequential data called *higher-order messages*, pass higher-order messages, and/or decode and invoke an encoded process from a higher-order message. The semantics for this uses a built-in sequential type `Store(S)` to represent a process based on its type signature `S`.

Example 18 shows higher-order processes passing `helloworld` from Example 1. A complete program based on this example is given in Appendix A.2.

Example 18: Passing `helloworld` between higher-order processes

```

1  proc ho_sender :: | => Put(Store(|Console=>)|TopBot) =
2      | => ch -> do
3          on ch do
4              put store(helloworld)    -- encode helloworld and send as a message
5              halt
6
7  proc ho_receiver :: | Put(Store(|Console=>)|TopBot), Console => =
8      | ch, console => -> do
9          on ch do
10             get stored_process        -- receive message with encoded helloworld
11             close
12         on console do
13             hput ConsolePut
14             put ("Higher order receiver says: Running the stored process")
15             use(stored_process)( | console => )    -- decode and invoke helloworld

```

Observe that `ho_sender` and `ho_receiver` are connected along a `Put(Store(|Console=>)|TopBot)` type channel. The `Store` type indicates that a higher-order message is being passed. The type signature of the encoded process, in this case `helloworld`, is indicated within the `Store` type.

To encode a process, the built-in `store` function is used. This function takes either the name of a previously defined process, as in the above example on line 4, or it takes an anonymous process as an in-line process definition.

To decode and invoke an encoded process, the `use` process command is used as in the above example on line 15. To `use` an encoded process, a higher-order process must provide instances of variables and channels of the correct types and polarities as specified by the `Store` type signature.

4 Sequential tier

The sequential tier of CaMPL is a basic functional programming language. Custom sequential types can be defined as data or codata. The value of variables can be used to conditionally branch the execution of functions and processes. This section discusses these features, which are related to the sequential type system. We synthesize content from [Kum18; Pon21; Pon22].

4.1 Functions

Functions are defined using the `fun` keyword, and they are called using their name and a comma-separated list of their input arguments enclosed in parentheses. We define a function `isEmpty` that will take a list as input and return a `Bool` depending on whether the input list is empty or not. We will use `isEmpty` in Example 24 to check whether the user has input an empty string.

Example 19: Sequential function that checks if a list is empty

```
1 fun isEmpty :: [A] -> Bool =  
2   [] -> True  
3   _  -> False
```

We can define infix functions such as concatenation `++`, which we used on line 30 of Example 15 `'put "message from user: " ++ msg'`:

Example 20: Infix concatenation function

```
1 fun (++) :: [A], [A] -> [A] =  
2   a, [] -> a  
3   [], a -> a  
4   (b:bs), cs -> b : (bs ++ cs)
```

The parentheses around the function name indicate an infix function is being defined. The name of an infix function is required to contain only special characters with some constraints [Fox23a].

CaMPL also supports mutually recursive function definitions and local function definitions. We demonstrate a local function definition in Example 23. This again uses `defn` and `where` in the same way as processes, and we describe the details for processes in Section 5.2.

4.2 Data and codata types

Built-in sequential types include `Int`, `Bool`, `Char`, tuples (e.g., the unit `()` or pairs `(Int, Bool)`), lists (e.g., strings `[Char]`), and higher-order messages (e.g., `Store(Int | Console=>TopBot)`). We explained and demonstrated the `Store` type in Section 3.4.

Custom sequential types can be defined as either `data` or `codata`. Data types are defined with constructors that build instances of their data structures. The data in a data structure can be accessed using `case` statements. Codata types allow one to represent potentially infinite structures that are evaluated lazily using their *destructors*. Instances of codata structures are built using *records* that define an implementation of the destructors for that instance.

Example 21 defines a “success or failure” SF data type (a.k.a. “Maybe”) and a `Stack` codata type which can optionally be implemented to be infinitely popped.

Example 21: Definition of a Stack codata type using a success/failure data type

```

1 data SF(A) -> C =
2   SS :: A -> C           -- Success constructor (a.k.a Just)
3   FF :: -> C             -- Failure constructor (a.k.a Nothing)
4
5 codata S -> Stack(A) =
6   Push :: A, S -> S       -- Push destructor
7   Pop  :: S -> (SF(A), S) -- Pop destructor uses SF data type

```

A Stack stores data of any sequential type A, and it has two destructors namely Push and Pop. New data can be added to the top of the Stack using Push. Pop removes the data at the top of the stack and returns a pair of that data, if it exists, and the resulting Stack. The SF data type allows a Pop to safely fail if the Stack is empty.

An instance of a Stack can be built from a list using (:) to implement the destructors. We define the following function listStack which returns a record that builds a Stack:

Example 22: Function that returns a record to build a Stack from a list

```

1 fun listStack :: [A] -> Stack(A) =
2   cs -> ( -- given a list, returns the record:
3     Push := c -> listStack(c:cs), -- implementation of Push
4     Pop  := -> case cs of         -- implementation of Pop
5       b:bs -> (SS(b), listStack(bs)) -- pop the top of the stack
6       [] -> (FF, listStack([]))    -- fail if the stack is empty
7   )

```

We use a Stack in the following reverse_print process to collect messages as they are received. Once all the messages have been received, we will flatten and print the stack from top to bottom. Since the most recent message will be on the top of the stack, the messages will be printed in reverse order.

Example 23: Printing messages from most recent to least recent

```

1 defn
2   proc reverse_print :: Stack([Char]) | SendMsgs([Char] | ), Console => =
3     stack | source, console => -> do
4       hcase source of
5         SendMsg -> do -- push messages to stack as they are received
6           get msg on source
7           reverse_print(Push(msg, stack) | source, console => )
8         CloseCh -> do -- after receiving all messages, print flattened stack
9           close source
10          on console do
11            hput ConsolePut
12            put "Printing stack from top to bottom: " ++ flatten(stack)
13            hput ConsoleClose
14            halt
15 where
16   fun flatten :: Stack([Char]) -> [Char] =
17     s -> case Pop(s) of -- recursively pop strings and concat them

```

```

18 (SS(str), stack) -> str ++ " " ++ flatten(stack)
19 (FF, stack) -> ""

```

Note that we don't actually require that `stack` is implemented using the `listStack` record. To use that implementation, we must initialize `stack` as `listStack([])` when `reverse_print` is first called. A complete program based on this example is given in [Appendix A.7](#).

Although CaMPL doesn't have built-in syntax for function types, which are required to write higher-order functions, we can define a function codata type, called `Func`, with an evaluation destructor, called `Eval`. A record that produces a `Func` instance defines the function as the implementation of `Eval`. This allows one to write higher-order functions, such as folds and maps. We define a function codata type and demonstrate a record that defines a function instance in the following code snippet:

```

1 codata F -> Func(A, B) =           -- function codata type
2   Eval :: A, F -> B                 -- Eval implementation defines each function
3
4 fun sumInts :: -> Func((Int, Int), Int) = -- produces a Func instance
5   -> (Eval := (a, b) -> a + b)       -- using Eval will call the function

```

4.3 Sequential control

Recall that, in [Example 15](#), client transmitted precisely one message from the user to server. To allow a user to input arbitrarily many messages before pressing enter to close the terminal, a stopping condition based on the sequential variable `msg` must control the client's execution. Conditional branching of a process's execution uses `if-then-else`, `case`, or `switch` statements or multiple process body definitions that pattern match on the value of variables.

We modify `client` to prompt the user for input until the user presses enter without typing a message, thereby sending an empty string. Each time `client` receives a string from the user, it will check if it is empty using the `isEmpty` function defined in [Example 19](#).

Example 24: Client execution conditional on user input

```

1 proc client :: | => SendMsgs([Char]| ), Terminal =
2   | => ch, terminal -> do
3     on terminal do           -- receive msg from user
4       hput StringTerminalPut
5       put "Hello User! Enter message in terminal. Press ENTER to close."
6       hput StringTerminalGet
7       get msg
8     if isEmpty(msg)          -- conditional branching on the value of msg
9       then do                -- user pressed ENTER, so close channels and halt
10        on ch do
11          hput CloseCh
12          close
13        on terminal do
14          hput StringTerminalClose
15          halt
16      else do                 -- user has sent a message, so forward to server

```

```

17     on ch do
18         hput SendMsg
19         put msg
20         client( | => ch, terminal)  -- recurse

```

A complete program based on this example is given in Appendix A.3.

The above example can also be implemented using **case** on the built-in constructors for Bool:

```

1  case isEmpty(msg) of
2      True -> do                -- user pressed ENTER, so close channels and halt
3          <lines 10-15>
4      False -> do               -- user has sent a message, so forward to server
5          <lines 17-20>

```

The above example can also be implemented using **switch**:

```

1  switch
2      isEmpty(msg) -> do        -- user pressed ENTER, so close channels and halt
3          <lines 10-15>
4      True -> do                -- user has sent a message, so forward to server
5          <lines 17-20>

```

Note that **switch** statements streamline the code for an **if-then-else if-then-...-else** statement rather than being a unique feature. That is, they execute the first process body in which the corresponding expression evaluates to True. Thus, a catch-all default branch can be defined using True as the expression.

The above example can also be implemented by defining another process to mutually recurse with client and conditionally branch using pattern matching on the instance of msg:

Example 25: Conditional branching using pattern matching

```

1  defn
2      proc client :: | => SendMsgs([Char]| ), Terminal =
3          | => ch, terminal -> do
4              on terminal do          -- receive msg from user
5                  hput StringTerminalPut
6                  put "Hello User! Enter message in terminal. Press ENTER to close."
7                  hput StringTerminalGet
8                  get msg
9                  pattern_match(msg | => ch, terminal)
10     proc pattern_match :: [Char] | => SendMsgs([Char]| ), Terminal =
11         "" | => ch, terminal -> do -- user pressed ENTER. close channels and halt
12             on ch do
13                 hput CloseCh
14                 close
15             on terminal do
16                 hput StringTerminalClose
17                 halt
18     msg | => ch, terminal -> do -- user sent a message, so forward to server
19         on ch do

```



```

20     hput SendMsg
21     put msg
22     client( | => ch, terminal)-- recurse

```

We discuss mutually recursive processes in Section 5.2.

5 Process commands manual

Conceptually, we have covered CaMPL's programming features in Sections 3 and 4. Table 2 summarizes the features and provides references to corresponding examples. The finer details of how one can successfully write programs using these features are provided explicitly in this section. We provide new examples of processes which coordinate access to a shared memory cell and use a (+) channel to **split** and **fork**. We will also revisit previous examples to emphasize the details of how one uses each process command.

5.1 Connecting processes using plug

We have seen the **plug** command used in most of our examples, especially in **run** processes. This command must be used as the last command in a process command block.

Recall the **run** process from Example 8 that connected broadcast to the source and destination processes:

```

1  proc run =
2    | => -> plug           -- plug command is the last (and only) command
3    sender( | => source)
4    broadcast("message broadcasted" | source => dest1, dest2)
5    send_to_dest1( | dest1, neg_dest1 => )
6    proc1( | => neg_dest1)
7    proc2( | dest2 =>)    -- proc2 cannot be connected to send_to_dest1

```

The **plug** command, used on line 2 in the above example, connects any number of processes along channels such that the network of connected processes remains acyclic. That is, the topology of a program is a connected finite acyclic graph consisting of processes as nodes and channels as edges. Processes which are plugged together run in parallel and may communicate via the channels connecting them.

Notice that each channel connects exactly two processes. Channels are linear resources which means that they cannot be arbitrarily created, duplicated, or destroyed. A channel can only be created in a **plug** command and each end must be plugged into a process. A channel cannot appear more than twice in a **plug** command either. Since the **plug** command is the last command in the command block, all existing open channels in scope must be passed to the processes being plugged. Sequential data is not a linear resource, so it can be duplicated, passed into, and used by any number of processes in a **plug** command.

Two processes can be plugged to each other if and only if they satisfy the following conditions: 1) both process definitions have a channel with the same type and opposite polarities, and 2) plugging the processes will not create a cycle. In the above example, **proc2** and **send_to_dest1**

Channel type	Description	Process command	Example
TopBot	Interaction on channel is over.	close or halt	6
Put(A Ch)	Message of type A travels from left to right (output to input polarity).	put/get	6
Get(A Ch)	Message of type A travels from right to left (input to output polarity).	get/put	6
Ch1(*)Ch2	Left process becomes two processes both connected to the right process.	fork/split	7
Ch1(+)Ch2	Right process becomes two processes both connected to the left process.	split/fork	29, 30
Neg(Ch)	Dual interaction of Ch by negating and identifying with another channel.	neg and =	8, 27
Custom protocol	Handles travel from left to right (output to input polarity).	hput/hcase	9, 11
Custom coprotocol	Handles travel from right to left (input to output polarity).	hcase/hput	12, 13
N/A	Connect pairs of processes by channels.	plug	2
Put(A Ch) or Get(A Ch)	Next process command block selected with controlled non-determinism.	race	16
N/A	Encode another process in a sequential Store type.	store	18
N/A	Decode and invoke the process encoded in a Store type.	use	18
N/A	Conditional branching.	if-then-else, case, switch	24

Table 2: Summary of channel types and process commands.

could not be connected because they both have input polarity channels but no output polarity channels. Furthermore, changing their process definitions and adding a channel is not possible either because they are both already connected to broadcast. Satisfying the second condition also requires that processes are not already connected to each other.

A **plug** command can plug previously defined processes, as above, and/or in-line process definitions. Recall Example 8, and notice the in-line process definitions on lines 7 - 8 and 9 - 12:

```

1  proc send_to_dest1 :: | Put([Char] | TopBot), Neg(Put([Char] | TopBot)) => =
2    | source, neg_dest1 => -> do
3      on source do
4        get msg
5        close
6      plug
7      neg_dest1, dest1 => ->      -- channels used by in-line proc definition

```

```

8      neg_dest1 |=| neg dest1 -- in-line process body
9      => dest1 ->              -- channels used by in-line proc definition
10     on dest1 do              -- in-line process body
11         put msg
12     halt

```

Notice that on lines 7 and 9, the input and output channels used by each process are given, but sequential variables are not given. Any sequential data in scope at line 6 can be used in the in-line process definitions.

5.2 Calling processes

A process can be invoked by calling it using its name and passing it valid instances of all of the variables and channels in its process definition. A process must be defined prior to its invocation unless the related processes are contained within a **defn** statement. Recall that we defined mutually recursive processes with **defn** in Example 25. Processes can also be locally defined using a **where** statement within a **defn** statement. Recall that we locally defined processes, which were also within a nested **defn** statement, in Example 16. Locally defined processes can only be called by the processes in the **defn** statement above the **where** keyword. For locally defined processes to call each other, as shown in Example 16, the nested **defn** statement on line 7 is required.

We have seen many examples with processes called in the **plug** command of a run process. When a process is called outside of a **plug** command, it must be the last command in the command block. Recall that we called `send_to_dest1` recursively in Example 13:

```

1  proc send_to_dest1 :: | SendMsgs([Char]| ), CoSendMsgs([Char]| ) => =
2      | source, dest1 => ->
3      hcase source of
4          SendMsg -> do
5              get msg on source          -- first line of the do block
6              on dest1 do
7                  hput CoSendMsg
8                  put msg
9              send_to_dest1( | source, dest1 => ) -- process call in the last line
10         CloseCh -> do
11             close source
12             on dest1 do
13                 hput CoCloseCh
14             halt

```

Notice that when we make the process call on line 9, it is the last command in the **do** block that starts on line 4. This means that all existing open channels in scope at the time the process is called must be passed to the process.

5.3 Equating channels using `id` and `neg`

The identification command, written as `|=|`, equates two channels of the same type and opposite polarities. This command can be thought of as calling an identity process that “does nothing” to

a channel. It must be used as the last command in a command block.

The `neg` command is used to equate two channels of the same polarity if one of them has a `Neg` type. The `neg` command changes a channel of type `X` to type `Neg(X)` and flips its polarity. Thus, after using `neg`, both channels will have type `Neg(X)` and opposite polarities, so they can be identified. The `neg` command can only be used within an identification command.

Negation and identification are particularly useful when channels are passed as resources between multiple processes. For example, consider the protocol `Passer` which coordinates exclusive access to a resource that can be instantiated using the concurrent type variable `R`:

Example 26: `Passer` protocol which uses a `Neg` channel

```
1 protocol Passer( |R) => S = -- coordinates access to a channel of type R
2   Pass :: R (+) Neg(S) => S -- uses a Neg channel to facilitate passing
```

We demonstrate `Passer` in Example 27 in which two processes take turns accessing a channel `mem` which connects them to a memory cell process called `memCell`. A complete version of this example in which the memory cell value is set with user input is available in Appendix A.8.

Example 27: Using `Passer` to pass a channel as a resource

```
1 proc run =
2   | => -> plug
3   memCell( "" | mem => ) -- memory cell is initially empty
4   memAccess("Ping" | passer => mem) -- Ping has access first
5   memWait("Pong" | => passer)
```

The processes are coordinating access to the memory cell `memCell`, and they want to write either `"Ping"` or `"Pong"`. They access the memory cell using channel `mem` which has type `MemCh([Char] |)`. The `memCell` code and `MemCh` protocol are straightforward and their definitions are given in the complete version.

The process with access to `memCell` runs `memAccess`. The process that is waiting runs `memWait`. When access to `memCell` changes, the processes will swap what they are running. That is, the process running `memAccess` will call `memWait` and vice versa. To define mutually recursive processes like this, we should use a `defn` statement, as mentioned in Section 5.2. However, for clearer explanations, we will consider the process definitions separately.

First, we will consider `memAccess`:

```
1 proc memAccess :: [Char] | Passer( |MemCh([Char] | )) => MemCh([Char] | ) =
2   tag | passer => mem -> do -- note: waiting process has other end of passer
3     on mem do -- access memory cell using mem
4       hput MemGet
5       get stored_tag
6       hput MemPut
7       put tag
8     hcase passer of Pass -> do -- block until other process requests to swap
9       fork passer as -- pass memory cell access and swap to waiting
10        pass_mem with mem ->
11        pass_mem |=| mem -- use id to pass mem access
12        neg_passer -> plug -- now wait on other end of passer
```

```

13     neg_passer, new_passer => ->
14     neg_passer |= neg new_passer -- negate other end to id with it
15     memWait(tag | => new_passer)  -- recurse with other end

```

We demonstrate how identification can be used to pass a channel as a resource on line 11. We demonstrate how a channel can be identified with a Neg channel using `neg` on line 14.

Next, consider `memWait`:

```

1  proc memWait :: [Char] | => Passer( | MemCh([Char] | )) =
2    tag | => passer -> do
3      hput Pass on passer -- request to swap to accessing
4      split passer into mem, neg_passer -- block until other proc passes mem
5      plug -- need to use other end of passer
6      => neg_passer, new_passer ->
7      neg_passer |= neg new_passer -- negate other end to id with it
8      memAccess(tag | new_passer => mem) -- recurse with other end

```

Again, we use `neg` to identify with a Neg channel on line 7.

5.4 Ending communication using `halt` and `close`

The `close` and `halt` commands terminate the interaction along a channel and remove it from scope. These commands are the only way a channel can be destroyed, and they can only be used on a channel with type `TopBot`. Recall process broadcast from Example 3. We can reorder the process commands without changing the functionality of the process as follows:

```

1  proc broadcast :: [Char] | Put([Char]|Get([Char]|TopBot))
   => Put([Char]|TopBot), Put([Char]|TopBot) =
2    ack_msg | source => dest1, dest2 -> do
3      get msg on source
4      put msg on dest1 -- the type of dest1 is TopBot after this line
5      close dest1 -- close can be used on a TopBot channel at any point
6      put msg on dest2
7      close dest2
8      put ack_msg on source
9      halt source -- halt is used to close the last channel and halt

```

Notice that the `close` command cannot be used as the last command in a command block, but it can be used at any other point to close `TopBot` channels. The `halt` command must be used as the last command in a command block because it halts a process after closing the last `TopBot` channel.

5.5 Complementary pairs

Complementary pairs of process commands are used on opposite polarities of a channel with a certain type. That is, two processes connected along a channel will each use one command in a complementary pair to perform their role in the interaction. An extended discussion on the importance of channel polarities is given in Section 3.2.1.

Each complementary pair has a *blocking* command and a *non-blocking* command. The process using the blocking command must wait until the other process performs the complementary

command to unblock it. For example, `get` is a blocking command, so a process is blocked until a message is received from the process performing the complementary `put` which sends the message. The process using `put` is not blocked, even if the other process hasn't received the message, because message passing is asynchronous. Table 3 summarizes complementary pairs of commands and whether the process that uses each command is blocked.

Command	Description
<code>put</code> <code>get</code>	Sends a value on a channel. Blocks until a value is received on a channel.
<code>hput</code> <code>hcase</code>	Sends a handle on a channel. Blocks until a handle is received on a channel.
<code>fork</code> <code>split</code>	Creates two new processes and channels (halts original channel). Blocks until other ends of new channels have processes (closes original channel).

Table 3: Complementary pairs of process commands.

5.5.1 Sending and receiving messages using `put` and `get`

We have used the `put` and `get` process commands to send messages and receive messages, respectively, in most of our examples so far. Recall Example 6 in which two processes are connected along a channel `ch`:

```

1  proc client =
2    | => ch ->           -- uses ch with output polarity
3    on ch do
4      put "Hello Server!" -- client sends string which will unblock server
5      get int
6      halt
7
8  proc server =
9    server_id | ch => ->  -- uses ch with input polarity
10   on ch do
11     get msg             -- server waits here until msg is received
12     put server_id
13     halt

```

The `get` command on line 11 blocks server until a value is received on channel `ch`. When the value is received, it binds the value to the variable `msg` and proceeds down the process command block. Channel `ch` is *used* in the `get` command, and hence, it must be in scope. The `get` command cannot be the last command in a command block.

The `put` command on line 4 sends the value `"Hello Server!"` on channel `ch` and proceeds down the process command block. This will allow the server process to unblock from its complementary `get` command on line 11. Again, `ch` is used in the `put` command so it must be in scope, and the `put` command cannot be the last command in a command block.

These commands can be used on channels with type `Put` or `Get` depending on the direction the message is travelling on the channel. Recall that messages travel on `Put` channels from left to right

(output to input polarity), and messages travel on **Get** channels from right to left (input to output polarity).

If the type of a channel is not explicitly defined, using **put** on the output polarity end and **get** on the input polarity end will force the type of the channel to be **Put**. Dually, using **get** on the output polarity end and **put** on the input polarity end will force the type of the channel to be **Get**. In the above example, the type of channel `ch` is **Put** when the string "Hello Server!" is sent and **Get** when the integer `server_id` is sent. Table 4 describes the usage of **put** and **get** process commands on a channel based on its type and polarity.

(a):	Type	Polarity	Command	(b):	Type	Polarity	Command
	Put	output	put		Get	output	get
	Put	input	get		Get	input	put

Table 4: Usage of **put** and **get** process commands for (a): **Put** and (b): **Get** types.

5.5.2 Channel activation using `hput` and `hcase`

Custom channel types, called protocols and coprotocols, allow processes to change the type of a channel dynamically at run-time. Recall the protocol `SendMsgs`, coprotocol `CoSendMsgs`, and modified `send_to_dest1` process from Examples 9, 12 and 13:

```

1 protocol SendMsgs (A | ) => S =
2   SendMsg :: Put(A | S) => S
3   CloseCh :: TopBot => S
4
5 coprotocol S => CoSendMsgs (A | ) =
6   CoSendMsg :: S => Get(A | S)
7   CoCloseCh :: S => TopBot
8
9 proc send_to_dest1 :: | SendMsgs([Char] | ), CoSendMsgs([Char] | ) => =
10  | source, dest1 => ->
11    hcase source of           -- receive protocol handle from broadcast
12      SendMsg -> do
13        get msg on source
14        on dest1 do
15          hput CoSendMsg      -- send coprotocol handle to dest1
16          put msg
17          send_to_dest1( | source, dest1 => ) -- recurse
18      CloseCh -> do
19        close source
20        on dest1 do
21          hput CoCloseCh
22          halt

```

The **hput** command on line 15 *activates* channel `dest1` by setting its type according to the handle `CoSendMsg` and proceeds down the process command block. Since `dest1` is used in the **hput**

command, it must be in scope. The `hput` command cannot be the last command in a command block. Since protocols and coprotocols are concurrent analogues of inductive and coinductive data types, we can think of `hput` as constructing the handle on the channel. We can also think of the handle as a session type and that the `hput` command initializes a session.

The `hcase` command on line 11 blocks `send_to_dest1` until a handle is received on channel `source`. When the handle is received, `send_to_dest1` proceeds according to the corresponding process command block, i.e. either lines 12 - 17 or lines 18 - 22. Since `source` is used in the `hcase` command, it must be in scope. All variables and channels that are in scope at line 11 are also in scope for each command block. The `hcase` command must be the last command in a command block. This means that all existing open channels in scope, i.e. `source` and `dest1`, must be closed or passed into other processes by the end of each block.

The handles used on a channel in a `hput` or `hcase` must be defined with the channel's type. Whether a process uses `hput` or `hcase` on a channel depends on its polarity and if the channel type is a protocol or coprotocol. Recall that handles travel on a protocol channel from left to right (output to input polarity which is the same as `Put`), and handles travel on a coprotocol channel from right to left (input to output polarity which is the same as `Get`). Since `send_to_dest1` uses the protocol channel `source` with input polarity, it receives handles using `hcase`. It also uses the coprotocol channel `dest1` with input polarity, so it sends handles using `hput`.

If the type of a channel is not explicitly defined, using the handles of an existing protocol in an `hput` command on the output polarity end and an `hcase` command on the input polarity end will force the channel type to be that protocol. This would not work if the handles belong to an existing coprotocol; in that case, the `hput` and `hcase` commands would need to be used on the opposite ends.

5.5.3 Multi-process communication using fork and split

The `fork` and `split` commands are used to make changes to the network of processes at run-time while ensuring no cycles are introduced. In Example 7, we used these commands to `fork` the process on the left end of a channel into two processes that were both connected to the process on the right. We will consider another example using these commands in which the process on the left uses `split` to talk to two new processes on the right instead.

Recall process broadcast from Example 3. We will consider the corresponding sender process that is connected along the source channel:

Example 28: Process that sends messages to broadcast

```
1 proc sender =  
2   | => source ->  
3   on source do  
4     put "Hi everyone!"  
5     get ack_msg  
6     halt
```

Suppose we want sender to first send a message that both destination processes will receive and then split the channel and send different messages to each process. We will omit acknowledgement messages in this modified example for simplicity.

Example 29: Splitting the source channel

```
1  proc sender =
2    | => source -> do
3      on source do
4        put "Hi everyone!"           -- msg sent to both procs
5        split source into source1, source2  -- unbundle channels
6        on source1 do                -- first send msg to 1
7          put "Hello proc1"
8          close
9        on source2 do                -- then send msg to 2 and halt
10         put "Hello proc2"
11         halt
```

The `split` command on line 5 removes the channel `source` from scope, creates two new channels `source1` and `source2` in scope, and blocks `sender` until two new processes are created and connected on the new channels. After the `split` command unblocks, `sender` will proceed down the process command block, which means the process commands used on the two new channels in lines 6 - 8 and 9 - 11 will be executed sequentially. The `source` is used in the `split` command, so it must be in scope at line 5, but the command removes it from scope. The two new channels will have the same polarity as the original channel. The `split` command cannot be the last command in a command block.

The corresponding broadcast process will need to `fork` its source channel and create two new processes to concurrently broadcast the second round of messages:

Example 30: Forking the source channel

```
12 proc broadcast =
13   ack_msg | source => dest1, dest2 -> do
14     get msg1 on source
15     put msg1 on dest1           -- broadcast msg1 to both processes
16     put msg1 on dest2
17     fork source as              -- unbundle channels
18       source1 with dest1 -> do  -- send msg2a to 1
19         get msg2a on source1
20         on dest1 do
21           put msg2a
22           close
23         halt source1
24       source2 with dest2 -> do  -- send msg2b to 2
25         get msg2b on source2
26         on dest2 do
27           put msg2b
28           close
29         halt source2
```

The `fork` command on line 17 removes the channel `source` from scope and creates two new channels that are each passed into one of the two process bodies on lines 18 - 23 and 24 - 29. After the `fork` command, the two process bodies each have one of the two new channels in scope and process

commands in each process body will be executed concurrently. Channel source is used in the **fork** command, so it must be in scope at line 17, but it is removed from scope by the command. The two new channels will have the same polarity as the original channel.

The **fork** command must be the last command in a command block. This means that all existing open channels in scope at line 17, i.e. `dest1` and `dest2`, must be passed into the process bodies. On line 18, `dest1` is passed into the first process body, and on line 24, `dest2` is passed into the second process body. All variables that are in scope when the **fork** command is made are also in scope for each process body. For example, if this process was sending acknowledgement messages, the variable `ack_msg` could be used in both process bodies.

These commands can be used on channels with type `(*)`, “tensor,” or `(+)`, “par,” depending on which end of the channel has one process **fork** into two processes. Recall that a `(*)` channel means the process on the left forks (using the channel with output polarity), and a `(+)` channel means the process on the right forks (using the channel with input polarity).

Again, if the type of a channel is not explicitly defined, using **fork** on the output polarity end and **split** on the input polarity end will force the type of the channel to be `(*)`. Dually, using **split** on the output polarity end and **fork** on the input polarity end will force the type of the channel to be `(+)`. In Example 29, the type of source is `(+)` because sender is on the output polarity end and broadcast is on the input polarity end. Table 5 describes the usage of the **fork** and **split** commands on a channel based on its type and polarity.

	Type	Polarity	Command		Type	Polarity	Command
(a):	<code>(*)</code>	output	fork	(b):	<code>(+)</code>	output	split
	<code>(*)</code>	input	split		<code>(+)</code>	input	fork

Table 5: Usage of **fork** and **split** process commands for (a): `(*)` and (b): `(+)` types.

5.6 Controlled non-determinism using race

The **race** command can be used to race channels on which a blocking command (see Section 5.5) is the next command that will be used on each channel. For example, if a **get** command is the next command that will be used on input polarity **Put** channels or output polarity **Get** channels, those channels can be used in a race just before any **get** commands are used. We recently added support for racing on **hcase** and **split** commands. Previous versions only supported **get** commands, so ensure the most recent version of the compiler is being used for full functionality.

A race allows the process to unblock when any channel in the race unblocks, e.g. when the first message is received on any of the channels in the race. We call this channel the *winner* of the race. When the process unblocks, it will execute the winner’s corresponding process body. The rest of the process bodies are ignored. Since only one of the race’s process bodies will be executed, all existing open channels in scope must be closed or passed into another process by the end of each process body. The **race** command must be the last command in a command block. We demonstrate the **race** command in the examples in Section 3.3.

5.7 Service channel handles

A current list of service channel handles supported by CaMPL is given in Example 31.

Example 31: Complete list of service channel handles

```
1 protocol Terminal => S =
2   StringTerminalPut :: Put([Char] | S) => S
3   StringTerminalGet :: Get([Char] | S) => S
4   StringTerminalClose :: TopBot => S
5
6   IntTerminalPut :: Put(Int | S) => S
7   IntTerminalGet :: Get(Int | S) => S
8   IntTerminalClose :: TopBot => S
9
10  CharTerminalPut :: Put(Char | S) => S
11  CharTerminalGet :: Get(Char | S) => S
12  CharTerminalClose :: TopBot => S
13
14 coprotocol S => Console =
15   ConsolePut :: S => Get([Char] | S)
16   ConsoleGet :: S => Put([Char] | S)
17   ConsoleClose :: S => TopBot
18   ConsoleStringTerminal :: S => S (*) Neg(Terminal)
19
20   IntConsolePut :: S => Get(Int | S)
21   IntConsoleGet :: S => Put(Int | S)
22   IntConsoleClose :: S => TopBot
23
24   CharConsolePut :: S => Get(Char | S)
25   CharConsoleGet :: S => Put(Char | S)
26   CharConsoleClose :: S => TopBot
27
28 coprotocol S => Timer =
29   Timer :: S => Get(Int | S (*) Put(()) | TopBot)
30   TimerClose :: S => TopBot
```

Notice that the handle `ConsoleStringTerminal` on line 18 allows one to **split** a `Console` channel to generate arbitrarily many `Terminal` channels. We have included a Prelude file in Appendix A.1 that provides service channel definitions. A modification to the compiler that will remove the necessity of user-defined services is currently in progress.

6 Mathematical underpinning

CaMPL implements a type system based on linear logic that was described in Cockett and Pastro’s paper “The Logic of Message Passing” [CP09]. The primary result of their paper was defining and proving a concurrent analogue of the functional programming Curry-Howard-Lambek correspondence or proofs-as-programs principle [How80; Lam69; Lam72]. They designed a two-tiered

logic consisting of a sequential tier, called the message logic, that interacts with a concurrent tier, called the message-passing logic. The message logic is a simple logic that corresponds to sequential functional programming. The message-passing logic corresponds to programming concurrent processes which use asynchronous message passing along channels. They provided the term calculus for both logics [CP09, Section 2.1, Section 3.1] which are implemented by CaMPL. We will discuss the syntax of CaMPL in reference to the two-tiered logic after briefly describing the categorical semantics of message passing.

The categorical semantics of message passing is given by a *linear actegory* in which a category of messages and sequential functions acts on a category of channels and concurrent processes in two directions. The covariant action passes messages in the forward direction, which we described in this article as “left to right,” and the contravariant action passes messages in the backward direction, which we described as “right to left.”

6.1 Categorical semantics

A linear $\mathbb{A}$ -actegory minimally consists of a monoidal category $\mathbb{A}$ acting covariantly and contravariantly on a linearly distributive category $\mathbb{X}$, satisfying certain coherences [CP09; CS97b]. The categorical semantics of CaMPL is modelled by a symmetric linear $\mathbb{A}$ -actegory in which a distributive symmetric monoidal category $\mathbb{A}$, which defines the semantics of the sequential tier, acts on a symmetric linearly distributive category $\mathbb{X}$, which defines the semantics of the concurrent tier. Linearly distributive categories are the categorical semantics of multiplicative linear logic [CS97b; Sri21].

In the monoidal category $\mathbb{A}$, objects correspond to sequential types, or message types, and morphisms correspond to sequential functions. Since $\mathbb{A}$ is a distributive monoidal category, it has coproducts over which its tensor distributes. The tensor in $\mathbb{A}$ corresponds to tuples and the coproducts correspond to data types with multiple constructors.

In the linearly distributive category $\mathbb{X}$, objects correspond to concurrent types, or channel types, and morphisms correspond to concurrent processes. Since $\mathbb{X}$ is a linearly distributive category, it has two functors $\otimes$ and $\oplus$, corresponding to the multiplicative conjunction $\otimes$ and the multiplicative disjunction $\oplus$ of linear logic:

$$\otimes : \mathbb{X} \times \mathbb{X} \rightarrow \mathbb{X} \quad \text{and} \quad \oplus : \mathbb{X} \times \mathbb{X} \rightarrow \mathbb{X}$$

These functors define the bundled channel types. The $\otimes$ functor corresponds to the $(*)$ channel type, and the $\oplus$ functor corresponds to the $(+)$ channel type.

The interaction of $\mathbb{A}$ and $\mathbb{X}$ is defined by two *action* functors:

$$\circ : \mathbb{A} \times \mathbb{X} \rightarrow \mathbb{X} \quad \text{and} \quad \bullet : \mathbb{A}^{\text{op}} \times \mathbb{X} \rightarrow \mathbb{X}$$

These functors describe how messages are passed on channels. The $\circ$ action corresponds to the **Put** channel type, and the $\bullet$ action corresponds to the **Get** channel type. The $\circ$ functor is the left parametrized left adjoint of $\bullet$ in the sense that the following is a parametrized adjunction for all message types A in $\mathbb{A}$:

$$A \circ - \dashv A \bullet - : \mathbb{X} \rightarrow \mathbb{X}$$

The adjunction signifies that a process sending or receiving a message can do so from either end of a channel.

The categorical semantics of CaMPL differ slightly from what was described in Cockett and Pastro’s paper. Since **TopBot** is the unit channel type, we only have one unit instead of two, so $\mathbb{X}$ is actually an isomix category [CS97a]. Furthermore, the **Neg** channel type gives each channel type a linear adjoint which means that $\mathbb{X}$ is a *-autonomous category [Bar91]. The categorical semantics of non-recursive protocols are coproducts, and coprotocols are products [CS01; CP04]. For recursive protocols (and coprotocols), the categorical semantics is given by pairs of initial functor algebras and final functor coalgebras [Yea12]. The categorical semantics for non-deterministic processes is obtained by enriching the concurrent semantics, $\mathbb{X}$, in sup-lattices by taking the power set of each hom-set of morphisms [Lit22]. This allows a process to be represented by a set of processes that define its possible executions. The categorical semantics for higher-order processes is obtained by enriching the concurrent semantics, $\mathbb{X}$, in the sequential semantics, $\mathbb{A}$ [Nor25]. This allows a process to be represented by a sequential type so that it can be passed as a (higher-order) message.

6.2 Two-tiered logic

The two-tiered logic gives a type system for concurrent processes that use message passing as their concurrency primitive [CP09]. Its inference rules govern operations within each tier and interactions between the two tiers. The two tiers provide a clean separation of computation and communication operations, so the complexities associated with each can be addressed effectively.

6.2.1 The logic of messages

The logic of messages, **Msg**, is the sequential tier, and the proofs in this logic correspond to sequential programs. It is concerned with the generation of messages and represents the logic of computation. The logic is presented in a Gentzen-style sequent calculus. A sequent takes the form

$$\Phi \vdash A$$

where the antecedent (or the *context*), Φ , is a comma-separated list of formulas and the succedent A is a single formula. The inference rules for **Msg** are given in Figure 6. In the inference rules, “subs” stands for substitution and is the sequential cut rule which corresponds to function composition.

6.2.2 The logic of message passing

The logic of message passing, **PMsg**, is the concurrent tier, and the proofs in this logic correspond to concurrent programs. It is concerned with interactions between processes over channels and represents the logic of communication. A sequent of **PMsg** corresponds to a process, so it has three components all of which are unordered lists: Φ is the sequential context of message types, Γ is input polarity channels types, and Δ is output polarity channel types. A sequent in **PMsg** is of the following form:

$$\Phi \mid \Gamma \Vdash \Delta$$

$$\begin{array}{c}
\text{axiom} \frac{}{\Phi \vdash A} \\
*_l \frac{\Phi, A, B \vdash C}{\Phi, A * B \vdash C} \\
I_l \frac{\Phi \vdash A}{\Phi, I \vdash A} \\
\text{coprod} \frac{\Phi, A \vdash C \quad \Phi, B \vdash C}{\Phi, A + B \vdash C} \\
0 \frac{}{\Phi, 0 \vdash A}
\end{array}
\qquad
\begin{array}{c}
\text{subs} \frac{\Phi \vdash A \quad \Psi_1, A, \Psi_2 \vdash B}{\Psi_1, \Phi, \Psi_2 \vdash B} \\
*_r \frac{\Phi \vdash A \quad \Psi \vdash B}{\Phi, \Psi \vdash A * B} \\
I_r \frac{}{\vdash I} \\
\text{inj}_l \frac{\Phi \vdash A}{\Phi \vdash A + B} \\
\text{inj}_r \frac{\Phi \vdash B}{\Phi \vdash A + B}
\end{array}$$

Figure 6: Inference rules for **Msg**.

The inference rules for **PMsg** are shown in Figure 7. Each rule corresponds to a process command. For example, the cut rule corresponds to the **plug** command. Complementary pairs of process commands arise from the two-sidedness of the inference rules.

The top half of the rules come from multiplicative linear logic and define the valid changes one can make to the process network. For example, we will consider the rules corresponding to **split** and **fork**. Recall that $\otimes$ and $\oplus$ correspond to the $(*)$ and $(+)$ types, respectively. The rule $\otimes_l$ is the same as using **split** on an input channel, and the rule $\otimes_r$ is the same as using **fork** on an output channel. The rule $\oplus_l$ is the same as using **fork** on an input channel, and the rule $\oplus_r$ is the same as using **split** on an output channel.

The bottom half of the rules include the action rules, which pertain to message passing, sequential context rules, and sequential control rules. For example, we will consider the rules corresponding to **get** and **put**. Recall that $\circ$ and $\bullet$ correspond to the **Put** and **Get** types, respectively. The rule $\circ_l$ is the same as using **get** on an input channel, and the rule $\circ_r$ is the same as using **put** on an output channel. The rule $\bullet_l$ is the same as using **put** on an input channel, and the rule $\bullet_r$ is the same as using **get** on an output channel.

Protocols, coprotocols, races, and higher-order message passing were not discussed in Cockett and Pasto’s paper. The inference rules for non-recursive protocols and coprotocols come from additive linear logic, which are similar to the additive rules in [CP04]. Recursive instances of protocols and coprotocols use proof boxes for fixed point combinators [Yea12]. Races do not change the type of a process, so an inference rule for a race only requires specifying the different possible executions as different orderings of inference rule applications depending on the outcome of the race. Higher-order message passing has two additional rules for **store** and **use** which are given in [Nor25].

7 Conclusion

We hope that a reader can now enthusiastically answer the questions “what was the type of channel ch from Example 2?” and “what is channel polarity?” Additionally, we hope that a reader was engaged by the examples we used to showcase the programming features of CaMPL. To maximize

$$\begin{array}{c}
\text{cut} \frac{\Phi \mid \Gamma_1 \Vdash \Delta_1, X \quad \Psi \mid X, \Gamma_2 \Vdash \Delta_2}{\Phi, \Psi \mid \Gamma_1, \Gamma_2 \Vdash \Delta_1, \Delta_2} \\
\\
\begin{array}{ll}
\text{atom id} \frac{}{\emptyset \mid X \Vdash X} & \text{axiom} \frac{}{\Phi \mid \Gamma \Vdash \Delta} \\
\otimes_l \frac{\Phi \mid \Gamma, X, Y \Vdash \Delta}{\Phi \mid \Gamma, X \otimes Y \Vdash \Delta} & \otimes_r \frac{\Phi \mid \Gamma_1 \Vdash \Delta_1, X \quad \Psi \mid \Gamma_2 \Vdash Y, \Delta_2}{\Phi, \Psi \mid \Gamma_1, \Gamma_2 \Vdash \Delta_1, X \otimes Y, \Delta_2} \\
\oplus_l \frac{\Phi \mid \Gamma_1, X \Vdash \Delta_1 \quad \Psi \mid Y, \Gamma_2 \Vdash \Delta_2}{\Phi, \Psi \mid \Gamma_1, X \oplus Y, \Gamma_2 \Vdash \Delta_1, \Delta_2} & \oplus_r \frac{\Phi \mid \Gamma \Vdash X, Y, \Delta}{\Phi \mid \Gamma \Vdash X \oplus Y, \Delta} \\
\top_l \frac{\Phi \mid \Gamma \Vdash \Delta}{\Phi \mid \Gamma, \top \Vdash \Delta} & \top_r \frac{}{\emptyset \mid \top \Vdash \Delta} \\
\perp_l \frac{}{\emptyset \mid \perp \Vdash \Delta} & \perp_r \frac{\Phi \mid \Gamma \Vdash \Delta}{\Phi \mid \Gamma \Vdash \perp, \Delta} \\
\\
\circ_l \frac{\Phi, A \mid \Gamma, X \Vdash \Delta}{\Phi \mid \Gamma, A \circ X \Vdash \Delta} & \circ_r \frac{\Phi \vdash A \quad \Psi \mid \Gamma \Vdash X, \Delta}{\Phi, \Psi \mid \Gamma \Vdash A \circ X, \Delta} \\
\bullet_l \frac{\Phi \vdash A \quad \Psi \mid \Gamma, X \Vdash \Delta}{\Phi, \Psi \mid \Gamma, A \bullet X \Vdash \Delta} & \bullet_r \frac{\Phi, A \mid \Gamma \Vdash X, \Delta}{\Phi \mid \Gamma \Vdash A \bullet X, \Delta} \\
* \frac{\Phi, A, B \mid \Gamma \Vdash \Delta}{\Phi, A * B \mid \Gamma \Vdash \Delta} & I \frac{\Phi \mid \Gamma \Vdash \Delta}{\Phi, I \mid \Gamma \Vdash \Delta} \\
\text{coprod} \frac{\Phi, A \mid \Gamma \Vdash \Delta \quad \Phi, B \mid \Gamma \Vdash \Delta}{\Phi, A + B \mid \Gamma \Vdash \Delta} & 0 \frac{}{\Phi, 0 \mid \Gamma \Vdash \Delta} \\
\\
\text{subs} \frac{\Phi \vdash A \quad \Psi, A \mid \Gamma \Vdash \Delta}{\Phi, \Psi \mid \Gamma \Vdash \Delta}
\end{array}
\end{array}$$

Figure 7: Inference rules for **PMsg**.

enjoyment of the examples, we suggest compiling and running them! The current implementation of CaMPL is available at <https://campl-ucalgary.github.io/>. A reasonably current version is available on the online compiler <https://campl-app.vercel.app/>. Complete programs that can be directly copied and pasted are given in Appendix A.

The CaMPL project is still very much in progress. As such, CaMPL is a proof-of-concept language for its categorical semantics. On the implementation side, we are working on updating the compiler to be more user friendly. We also want to add features to work with processes that are distributed over multiple devices and connected by a network. Finally, of particular interest and novelty is adding type classes to the concurrent type system of CaMPL. Recall that, in Haskell, the type class system increases the expressiveness of the language significantly by enabling adhoc polymorphism.

On the categorical semantics side, we are working on a precise semantics for non-determinism that considers how races interact with the other features. We are also considering a categorical semantics for message passing between quantum processes [CS23]. This will contribute to the development of programming languages for distributed quantum computing over a quantum internet.

References

- [Abr93a] Samson Abramsky. “Computational interpretations of linear logic”. In: *Theoretical Computer Science* 111.1 (1993), pp. 3–57 (cit. on p. 2).
- [Abr93b] Samson Abramsky. “Interaction Categories”. In: Jan. 1993, pp. 57–69 (cit. on p. 2).
- [AG19] Federico Aschieri and Francesco A. Genco. “Par means parallel: multiplicative linear logic proofs as concurrent functional programs”. In: *Proc. ACM Program. Lang.* 4.POPL (Dec. 2019) (cit. on p. 3).
- [AGN00] Samson Abramsky, Simon Gay, and Rajagopal Nagarajan. “Specification Structures and Propositions-as-Types for Concurrency”. In: *Lecture Notes in Computer Science* (Jan. 2000) (cit. on p. 2).
- [AM99] Samson Abramsky and Paul-André Melliès. “Concurrent games and full completeness”. In: *Proceedings. 14th Symposium on Logic in Computer Science (Cat. No. PR00158)* (1999), pp. 431–442 (cit. on p. 2).
- [Bar+97] Andrew G. Barber, Philippa Gardner, Masahito Hasegawa, and Gordon D. Plotkin. “From Action Calculi to Linear Logic”. In: *Computer Science Logic, 11th International Workshop, CSL ’97, Annual Conference of the EACSL, Aarhus, Denmark, August 23-29, 1997, Selected Papers*. Ed. by Mogens Nielsen and Wolfgang Thomas. Vol. 1414. Lecture Notes in Computer Science. Springer, Aug. 1997, pp. 78–97 (cit. on p. 3).
- [Bar91] Michael Barr. “*-Autonomous categories and linear logic”. In: *Mathematical Structures in Computer Science* 1.2 (1991), pp. 159–178 (cit. on p. 36).
- [BS94] G. Bellin and P.J. Scott. “On the π -calculus and linear logic”. In: *Theoretical Computer Science* 135.1 (1994), pp. 11–65 (cit. on p. 3).
- [CN26] Robin Cockett and Melika Norouzbeygi. “Actegories, Copowers, and Higher-Order Message Passing Semantics”. In: *Electronic Proceedings in Theoretical Computer Science* 442 (Mar. 2026), pp. 45–59. ISSN: 2075-2180. DOI: [10.4204/eptcs.442.4](https://doi.org/10.4204/eptcs.442.4). URL: <http://dx.doi.org/10.4204/EPTCS.442.4> (cit. on p. 3).
- [CP04] Robin Cockett and C. Pastro. “A Language For Multiplicative-additive Linear Logic”. In: *Electronic Notes in Theoretical Computer Science* 122 (Apr. 2004). DOI: [10.1016/j.entcs.2004.06.049](https://doi.org/10.1016/j.entcs.2004.06.049) (cit. on pp. 36, 37).
- [CP09] Robin Cockett and Craig Pastro. “The logic of message-passing”. In: *Science of Computer Programming* 74.8 (2009), pp. 498–533 (cit. on pp. 2, 3, 34–36).
- [CP10] Luís Caires and Frank Pfenning. “Session Types as Intuitionistic Linear Propositions”. In: *CONCUR 2010 - Concurrency Theory*. Ed. by Paul Gastin and François Laroussinie. Berlin, Heidelberg: Springer Berlin Heidelberg, 2010, pp. 222–236 (cit. on p. 3).
- [CS01] JRB Cockett and RAG Seely. “Finite sum-product logic”. In: *Theory and Applications of Categories* 8.5 (2001), pp. 63–99 (cit. on p. 36).
- [CS23] Robin Cockett and Priyaa Varshinee Srinivasan. *Quantum Message Passing Logic (Talk)*. https://www.reluctantm.com/gcruttw/fmcs2023/Slides/FMCS_2023_Day_2.pdf. Accessed: 2026-04-13. 2023 (cit. on p. 38).

- [CS97a] Robin Cockett and Robert Seely. “Proof theory for full intuitionistic linear logic, bilinear logic, and mix categories”. In: *Theory and Applications of categories* 3.5 (1997), pp. 85–131 (cit. on p. 36).
- [CS97b] Robin Cockett and Robert Seely. “Weakly distributive categories”. In: *Journal of Pure and Applied Algebra* 114.2 (1997), pp. 133–173 (cit. on pp. 3, 35).
- [Fox23a] Braden Foxcroft. *Notes on infix operators*. https://github.com/campl-ucalgary/campl/blob/main/resources/notes_on_infix_operators.txt. Accessed: 2026-04-29. 2023 (cit. on p. 20).
- [Fox23b] Braden Foxcroft. *Notes on modules*. https://github.com/campl-ucalgary/campl/blob/main/resources/notes_on_modules.txt. Accessed: 2026-05-16. 2023 (cit. on p. 42).
- [Gir87] Jean-Yves Girard. “Linear logic”. In: *Theoretical Computer Science* 50.1 (1987), pp. 1–101 (cit. on p. 2).
- [Hon93] Kohei Honda. “Types for Dyadic Interaction”. In: *International Conference on Concurrency Theory*. 1993 (cit. on p. 3).
- [How80] William A. Howard. “The Formulae-as-Types Notion of Construction [Original manuscript from 1969]”. In: *To H. B. Curry: Essays on Combinatory Logic, Lambda Calculus and Formalism*. Ed. by Jonathan P. Seldin and J. Roger Hindley. Academic Press, 1980, pp. 479–490. ISBN: 978-0-12-349050-6 (cit. on pp. 2, 34).
- [KMP19] Wen Kokke, Fabrizio Montesi, and Marco Peressotti. “Better late than never: a fully-abstract semantics for classical processes”. In: *Proc. ACM Program. Lang.* 3.POPL (Jan. 2019) (cit. on p. 3).
- [KPT96] Naoki Kobayashi, Benjamin Pierce, and David Turner. “Linearity and the Pi-Calculus”. In: *23rd Symposium on Principles of Programming Languages, POPL’96*. ACM, 1996, pp. 358–371 (cit. on p. 3).
- [Kum18] Prashant Kumar. “Implementation of Message Passing Language”. Master’s Thesis. Calgary, Alberta, Canada: University of Calgary, Feb. 2018. URL: <https://cspages.ucalgary.ca/~robin/Theses/PrashantKumar.pdf> (cit. on pp. 3, 5, 20).
- [Lam69] Joachim Lambek. “Deductive systems and categories II: Standard constructions and closed categories”. In: *Category Theory, Homology Theory and their Applications I*. Ed. by P. Hilton. Vol. 86. Lecture Notes in Mathematics. Springer, 1969, pp. 76–122 (cit. on pp. 2, 34).
- [Lam72] Joachim Lambek. “Deductive systems and categories III: Cartesian closed categories, intuitionist propositional calculus, and combinatory logic”. In: *Toposes, Algebraic Geometry and Logic*. Ed. by F. W. Lawvere. Vol. 274. Lecture Notes in Mathematics. Springer, 1972, pp. 57–82 (cit. on pp. 2, 34).

- [Lit22] Alexanna Little. “Semantics for Non-Determinism in the Categorical Message Passing Language”. PURE Final Assignment: Research Findings and Synthesis. Calgary, Alberta, Canada: University of Calgary, Sept. 2022. URL: https://github.com/campl-ucalgary/campl/blob/main/resources/PURE2022_ResearchFindingsSynthesis_Little.pdf (cit. on pp. 3, 36).
- [Lit23] Alexanna Little. “Formalizing Non-Determinism in the Categorical Message Passing Language”. Undergraduate Thesis. Calgary, Alberta, Canada: University of Calgary, Apr. 2023. URL: https://github.com/campl-ucalgary/campl/blob/main/resources/CPSC502F22W23_FinalReport_Little.pdf (cit. on p. 3).
- [Lyb18] Reginald Lybbert. “Progress for the Message Passing Logic”. Undergraduate Thesis. Calgary, Alberta, Canada: University of Calgary, Apr. 2018. URL: <https://github.com/campl-ucalgary/campl/blob/main/resources/ProgressForMPL.pdf> (cit. on p. 3).
- [Mil93] Robin Milner. “The Polyadic π -Calculus: a Tutorial”. In: *Logic and Algebra of Specification*. Ed. by Friedrich L. Bauer, Wilfried Brauer, and Helmut Schwichtenberg. Berlin, Heidelberg: Springer Berlin Heidelberg, 1993, pp. 203–246 (cit. on p. 3).
- [Nor25] Melika Norouzbeygi. “Higher-Order Message Passing in CaMPL”. Master’s Thesis. Calgary, Alberta, Canada: University of Calgary, Sept. 2025. URL: <https://cspages.ucalgary.ca/~robin/Theses/Melika.pdf> (cit. on pp. 3, 5, 36, 37).
- [Pon21] Jared Pon. “Implementation Status of CMPL”. Undergraduate Thesis Interim Report. Calgary, Alberta, Canada: University of Calgary, Dec. 2021. URL: https://github.com/campl-ucalgary/campl/blob/main/resources/502.02A_interim_pon.pdf (cit. on pp. 3, 5, 20).
- [Pon22] Jared Pon. “Redesigning the Abstract Machine for CaMPL”. Undergraduate Thesis. Calgary, Alberta, Canada: University of Calgary, Apr. 2022. URL: https://github.com/campl-ucalgary/campl/blob/main/resources/JaredPon_final_report.pdf (cit. on pp. 3, 5, 20).
- [QKB21] Zesen Qian, G. A. Kavvos, and Lars Birkedal. “Client-server sessions in linear logic”. In: *Proc. ACM Program. Lang.* 5.ICFP (Aug. 2021) (cit. on p. 3).
- [Sri21] Priyaa Varshinee Srinivasan. “Dagger linear logic and categorical quantum mechanics”. Available at <https://arXiv:2303.14231>. PhD thesis. Calgary, AB: University of Calgary, 2021 (cit. on p. 35).
- [TCP13] Bernardo Toninho, Luis Caires, and Frank Pfenning. “Higher-Order Processes, Functions, and Sessions: A Monadic Integration”. In: *Programming Languages and Systems*. Ed. by Matthias Felleisen and Philippa Gardner. Berlin, Heidelberg: Springer Berlin Heidelberg, 2013, pp. 350–369 (cit. on p. 3).
- [Wad12] Philip Wadler. “Propositions as sessions”. In: *Proceedings of the 17th ACM SIGPLAN International Conference on Functional Programming*. ICFP ’12. Copenhagen, Denmark: Association for Computing Machinery, 2012, pp. 273–286 (cit. on p. 3).

- [Yea12] Masuka Yeasin. “Linear Functors and their Fixed Points”. Master’s Thesis. Calgary, Alberta, Canada: University of Calgary, Dec. 2012. URL: https://cspages.ucalgary.ca/~robin/Theses/masuka_thesis.pdf (cit. on pp. 3, 36, 37).

A Examples: Complete programs

The current implementation of CaMPL is available at <https://campl-ucalgary.github.io/> – our website has detailed instructions on how to run CaMPL code. A reasonably current version is available on the online compiler <https://campl-app.vercel.app/>. These example programs, along with many others, are also available on our Github: <https://github.com/campl-ucalgary/campl/tree/main/MPLCLI/examples/complete-appendix-programs>

A.1 Prelude

CaMPL has a module system that allows one to include and use code defined in other files [Fox23b]. This Prelude.mpl file combines code from Examples 14, 19, 20, and 31. We encourage readers to write additional functions in their version of the Prelude as an exercise in CaMPL programming. We demonstrate how to include the Prelude in A.2, A.3, A.5, A.7, and A.8.

```
1  -- combines code from Examples 14, 31:
2  protocol Terminal => S =
3      StringTerminalPut :: Put([Char]|S) => S
4      StringTerminalGet :: Get([Char]|S) => S
5      StringTerminalClose :: TopBot => S
6      IntTerminalPut :: Put(Int|S) => S
7      IntTerminalGet :: Get(Int|S) => S
8      IntTerminalClose :: TopBot => S
9      CharTerminalPut :: Put(Char|S) => S
10     CharTerminalGet :: Get(Char|S) => S
11     CharTerminalClose :: TopBot => S
12
13 coprotocol S => Console =
14     ConsolePut :: S => Get([Char]|S)
15     ConsoleGet :: S => Put([Char]|S)
16     ConsoleClose :: S => TopBot
17     ConsoleStringTerminal :: S => S (*) Neg(Terminal)
18     IntConsolePut :: S => Get(Int|S)
19     IntConsoleGet :: S => Put(Int|S)
20     IntConsoleClose :: S => TopBot
21     CharConsolePut :: S => Get(Char|S)
22     CharConsoleGet :: S => Put(Char|S)
23     CharConsoleClose :: S => TopBot
24
25 coprotocol S => Timer =
26     Timer :: S => Get(Int|S (*) Put(())|TopBot))
27     TimerClose :: S => TopBot
28
```

```

29 -- from Example 19:
30 fun isEmpty :: [A] -> Bool =
31   [] -> True
32   _  -> False
33
34 -- from Example 20:
35 fun (++) :: [A],[A] -> [A] =
36   a,[] -> a
37   [],a -> a
38   (b:bs),cs -> b : (bs ++ cs)

```

A.2 Higher order Hello World

This program includes the Prelude from [A.1](#) and combines code from Examples [1](#) and [18](#).

```

1 include Prelude
2
3 -- from Example 1:
4 proc helloworld :: | Console => =
5   | console => -> do
6     hput ConsolePut on console
7     put "Hello World!" on console
8     hput ConsoleClose on console
9     halt console
10
11 -- from Example 18:
12 proc ho_sender :: | => Put( Store(|Console=>) | TopBot) =
13   | => ch -> do
14     on ch do
15       put store(helloworld)
16       halt
17
18 proc ho_receiver :: | Put( Store(|Console=>) | TopBot), Console => =
19   | ch, console => -> do
20     on ch do
21       get stored_process
22       close
23     on console do
24       hput ConsolePut
25       put ("Higher order receiver says: Running the stored process")
26       use(stored_process)( | console => )
27
28 -- new code:
29 proc run :: | Console => =
30   | console => -> plug
31     ho_sender( | => ch)
32     ho_receiver( | ch, console => )

```

A.3 Requesting user input continuously

This program includes the Prelude from A.1 and combines code from Examples 9, 15, 24.

```
1 include Prelude (isEmpty| )
2
3 -- from Example 9:
4 protocol SendMsgs (A| ) => S =
5     SendMsg :: Put(A|S) => S
6     CloseCh :: TopBot => S
7
8 -- from Example 24:
9 proc client :: | => SendMsgs([Char]|), Terminal =
10 | => ch, terminal -> do
11     on terminal do
12         hput StringTerminalPut
13         put "Hello User! Enter message in terminal. Press ENTER to close."
14         hput StringTerminalGet
15         get msg
16         if isEmpty(msg)
17         then do
18             on ch do
19                 hput CloseCh
20                 close
21             on terminal do
22                 hput StringTerminalClose
23                 halt
24         else do
25             on ch do
26                 hput SendMsg
27                 put msg
28             client( | => ch, terminal)
29
30 -- from Example 15:
31 proc server :: | SendMsgs([Char]|), Console => =
32 | ch, console => -> do
33     hcase ch of
34         SendMsg -> do
35             get msg on ch
36             on console do
37                 hput ConsolePut
38                 put "message from user: " ++ msg
39             server( | ch, console => )
40         CloseCh -> do
41             close ch
42             on console do
43                 hput ConsoleClose
44                 halt
45
```

```

46 proc run :: | Console => Terminal =
47   | console => terminal -> plug
48     client( | => ch, terminal)
49     server( | ch, console => )

```

A.4 Broadcasting a single message

Note that this program will not produce observable effects when it is run. This program combines code from Examples 3, 8, 28.

```

1  -- from Example 28:
2  proc sender :: | => Put([Char] | Get([Char] | TopBot)) =
3    | => source ->
4      on source do
5        put "Hi everyone!"
6        get ack_msg
7        halt
8
9  -- from Example 3:
10 proc broadcast :: [Char] | Put([Char] | Get([Char] | TopBot))
    => Put([Char] | TopBot), Put([Char] | TopBot) =
11   ack_msg | source => dest1, dest2 -> do
12     get msg on source
13     put msg on dest1
14     put msg on dest2
15     put ack_msg on source
16     close source
17     close dest1
18     halt dest2
19
20 -- from Example 8:
21 proc send_to_dest1 :: | Put([Char] | TopBot), Neg(Put([Char] | TopBot)) => =
22   | source, neg_dest1 => -> do
23     on source do
24       get msg
25       close
26     plug
27     neg_dest1, dest1 => ->
28     neg_dest1 |=| neg dest1
29     => dest1 ->
30     on dest1 do
31       put msg
32       halt
33
34 -- new code:
35 proc proc1 :: | => Neg(Put([Char] | TopBot)) =
36   | => neg_dest1 -> plug
37   => neg_dest1, dest1 ->

```

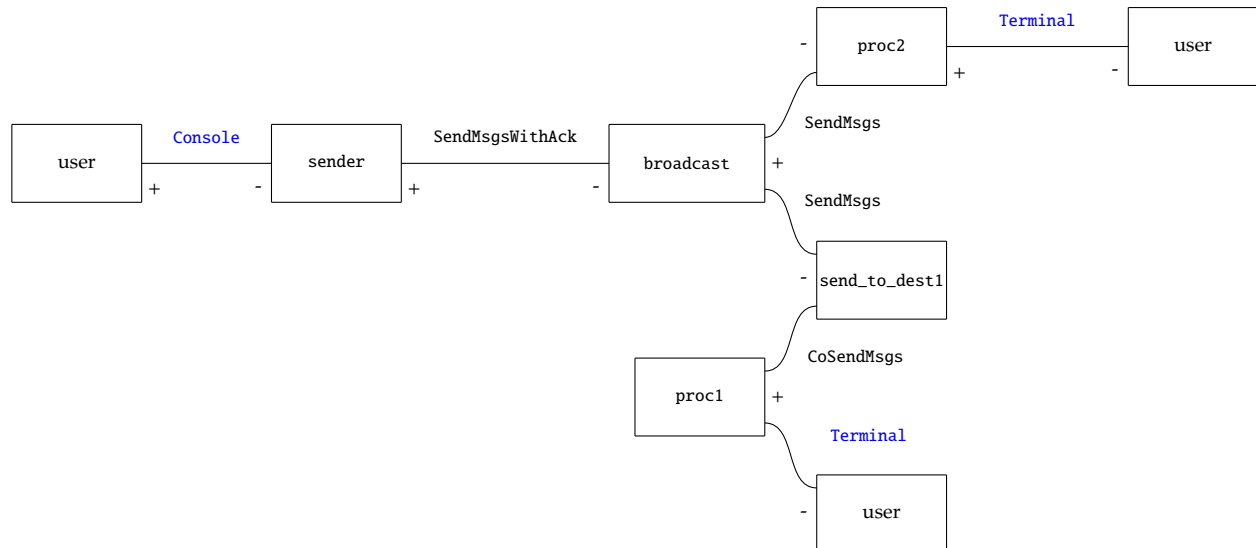
```

38     neg_dest1 |=| neg dest1
39     dest1 => ->
40     on dest1 do
41         get msg
42         halt
43
44 proc proc2 :: | Put([Char] | TopBot) => =
45 | dest2 => ->
46 on dest2 do
47     get msg
48     halt
49
50 -- from Example 8:
51 proc run =
52 | => -> plug
53 sender( | => source)
54 broadcast("message broadcasted" | source => dest1, dest2)
55 send_to_dest1( | dest1, neg_dest1 => )
56 proc1( | => neg_dest1)
57 proc2( | dest2 =>)

```

A.5 Broadcasting an arbitrary number of messages

This program includes the Prelude from A.1 and combines code from Examples 9, 10, 11, 12, 13. It also uses ideas from Examples 15 and 24. We visualize the process network for this program in the following diagram:



```

1 include Prelude (isEmpty | )
2
3 -- from Example 9:
4 protocol SendMsgs(A | ) => S =
5     SendMsg :: Put(A | S) => S

```

```

6   CloseCh :: TopBot => S
7
8   -- from Example 10:
9   protocol SendMsgsWithAck(A| ) => S =
10    SendMsgWithAck :: Put(A|Get([Char]|S)) => S
11    CloseAckCh :: TopBot => S
12
13   -- from Example 12:
14   coprotocol S => CoSendMsgs(A| ) =
15    CoSendMsg :: S => Get(A|S)
16    CoCloseCh :: S => TopBot
17
18   -- new code (combines ideas from Examples 15 and 24):
19   proc sender :: | Console => SendMsgsWithAck([Char]| ) =
20    | console => source -> do
21      on console do
22        hput ConsolePut
23        put "Hello User! Enter message to broadcast. Press ENTER to close."
24        hput ConsoleGet
25        get msg
26        if isEmpty(msg)
27        then do
28          on console do
29            hput ConsolePut
30            put "Indicating to destination processes that source is finished."
31          on source do
32            hput CloseAckCh
33            close
34          on console do
35            hput ConsoleClose
36            halt
37        else do
38          on source do
39            hput SendMsgWithAck
40            put msg
41            get ack_msg
42          on console do
43            hput ConsolePut
44            put ack_msg
45          sender( | console => source)
46
47   -- from Example 11:
48   proc broadcast :: [Char] | SendMsgsWithAck([Char]| ) => SendMsgs([Char]| ),
49    SendMsgs([Char]| ) =
50   ack_msg | source => dest1, dest2 -> do
51     hcase source of
52       SendMsgWithAck -> do
53         get msg on source
54         on dest1 do

```



```

54         hput SendMsg
55         put msg
56     on dest2 do
57         hput SendMsg
58         put msg
59     on source do
60         put ack_msg
61         broadcast(ack_msg | source => dest1, dest2)
62     CloseAckCh -> do
63         close source
64     on dest1 do
65         hput CloseCh
66         close
67     on dest2 do
68         hput CloseCh
69         halt
70
71 -- from Example 13:
72 proc send_to_dest1 :: | SendMsgs([Char]| ), CoSendMsgs([Char]| ) => =
73     | source, dest1 => ->
74     hcase source of
75         SendMsg -> do
76             get msg on source
77             on dest1 do
78                 hput CoSendMsg
79                 put msg
80             send_to_dest1( | source, dest1 => )
81         CloseCh -> do
82             close source
83             on dest1 do
84                 hput CoCloseCh
85                 halt
86
87 -- new code (combines ideas from Examples 15 and 24):
88 proc proc1 :: [Char] | => CoSendMsgs([Char]| ), Terminal =
89     tag | => source, terminal -> do
90         hcase source of
91             CoSendMsg -> do
92                 get msg on source
93                 on terminal do
94                     hput StringTerminalPut
95                     put tag ++ " received message: " ++ msg
96                     proc1(tag | => source, terminal)
97             CoCloseCh -> do
98                 close source
99                 on terminal do
100                     hput StringTerminalPut
101                     put "Source has finished sending messages to " ++ tag ++ ". Press
                        ENTER to close."

```

```

102     hput StringTerminalGet
103     get _
104     hput StringTerminalClose
105     halt
106
107 proc proc2 :: [Char] | SendMsgs([Char]| ) => Terminal =
108   tag | source => terminal -> do
109     hcase source of
110       SendMsg -> do
111         get msg on source
112         on terminal do
113           hput StringTerminalPut
114           put tag ++ " received message: " ++ msg
115           proc2(tag | source => terminal)
116       CloseCh -> do
117         close source
118         on terminal do
119           hput StringTerminalPut
120           put "Source has finished sending messages to " ++ tag ++ ". Press
121             ENTER to close."
122           hput StringTerminalGet
123           get _
124           hput StringTerminalClose
125           halt
126
127 proc run =
128   | console => term1, term2 -> plug
129     sender( | console => source)
130     broadcast("Message has been broadcasted." | source => dest1, dest2)
131     send_to_dest1( | dest1, codest1 => )
132     proc1("Process 1" | => codest1, term1)
133     proc2("Process 2" | dest2 => term2)

```

A.6 Non-deterministic server interacts with two clients

Note that this program will not produce observable effects when it is run. This program combines code from Examples 2, 7, 16.

```

1  -- from Example 16:
2  defn
3    proc server =
4      | two_ch => -> do
5        split two_ch into ch1, ch2
6        server_non_deterministic( | ch1, ch2 => )
7
8  where defn
9    proc server_non_deterministic =
10     | ch1, ch2 => -> do

```

```

11     race
12     ch1 -> server_deterministic( | ch1, ch2 => )
13     ch2 -> server_deterministic( | ch2, ch1 => )
14
15 proc server_deterministic =
16   | winner, loser => -> do
17     on winner do
18       get msg
19       put msg
20       close
21     on loser do
22       get msg
23       put msg
24       halt
25
26 -- from Example 2:
27 proc client =
28   | => ch ->
29     on ch do
30       put "Hello Server!"
31       get echo
32       halt
33
34 -- from Example 7:
35 proc two_clients =
36   | => two_ch ->
37     fork two_ch as
38       ch1 -> client( | => ch1)
39       ch2 -> client( | => ch2)
40
41 proc run :: | => =
42   | => -> plug
43     two_clients( | => two_ch)
44     server( | two_ch => )

```

A.7 Pushing messages onto a Stack codata type

This program includes the Prelude from [A.1](#) and combines code from Examples [9](#), [21](#), [22](#), [23](#), and [24](#).

```

1 include Prelude (isEmpty | )
2
3 -- from Example 9:
4 protocol SendMsgs(A | ) => S =
5   SendMsg :: Put(A | S) => S
6   CloseCh :: TopBot => S
7
8 -- from Example 21:

```

```

9  data SF(A) -> C =
10     SS :: A -> C
11     FF :: -> C
12
13  codata S -> Stack(A) =
14     Push :: A, S -> S
15     Pop  :: S -> (SF(A), S)
16
17  -- from Example 22:
18  fun listStack :: [A] -> Stack(A) =
19     cs -> (
20         Push := c -> listStack(c:cs),
21         Pop  := -> case cs of
22             b:bs -> (SS(b), listStack(bs))
23             [] -> (FF, listStack([]))
24         )
25
26  -- from Example 23:
27  defn
28     proc reverse_print :: Stack([Char]) | SendMsgs([Char]| ), Console => =
29     stack | source, console => -> do
30         hcase source of
31             SendMsg -> do
32                 get msg on source
33                 reverse_print(Push(msg, stack) | source, console => )
34             CloseCh -> do
35                 close source
36                 on console do
37                     hput ConsolePut
38                     put "Printing stack from top to bottom: " ++ flatten(stack)
39                     hput ConsoleClose
40                     halt
41  where
42     fun flatten :: Stack([Char]) -> [Char] =
43     s -> case Pop(s) of
44         (SS(str), stack) -> str ++ " " ++ flatten(stack)
45         (FF, stack) -> ""
46
47
48  -- from Example 24:
49  proc client :: | => SendMsgs([Char]|), Terminal =
50  | => ch, terminal -> do
51      on terminal do
52          hput StringTerminalPut
53          put "Hello User! Enter message in terminal. Press ENTER to close."
54          hput StringTerminalGet
55          get msg
56          if isEmpty(msg)
57          then do

```

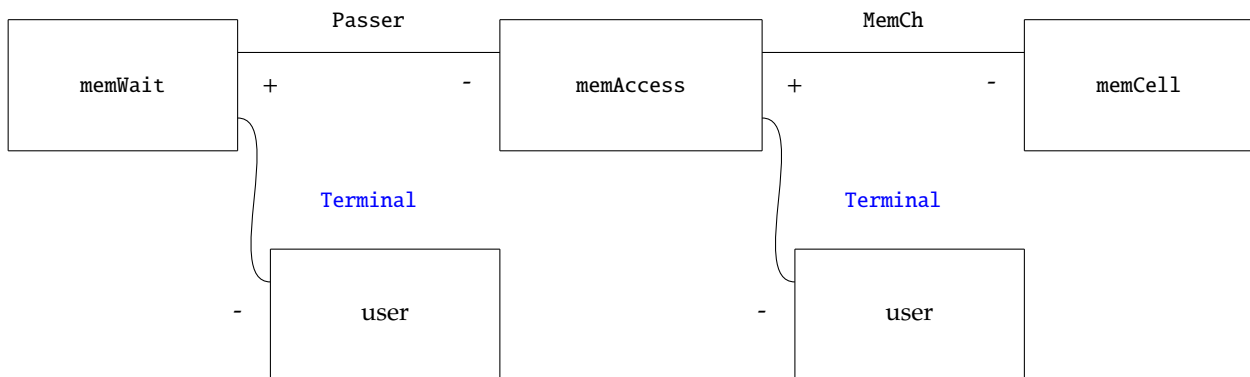
```

58     on ch do
59         hput CloseCh
60         close
61     on terminal do
62         hput StringTerminalClose
63         halt
64     else do
65         on ch do
66             hput SendMsg
67             put msg
68             client( | => ch, terminal)
69
70 -- new code:
71 proc server :: | SendMsgs([Char]| ), Console => =
72 | ch, console => -> do
73     on console do
74         hput ConsolePut
75         put "Collecting messages to print in reverse."
76         reverse_print(listStack([]) | ch, console => ) -- initializes stack
77
78 proc run :: | Console => Terminal =
79 | console => terminal -> plug
80     client( | => ch, terminal)
81     server( | ch, console => )

```

A.8 Memory cell

This program includes the Prelude from A.1 and modifies code from Examples 26 and 27. We visualize the process network for this program in the following diagram:



```

1 include Prelude (isEmpty| )
2
3 -- modified from Example 26:
4 protocol Passer( |R) => S =
5     Pass :: R (+) Neg(S) => S
6     Done :: TopBot => S
7
8 -- this handle is new

```

```

8  -- new code:
9  protocol MemCh(A | ) => S =                -- memory cell access protocol
10     MemPut  :: Put(A | S) => S              -- write
11     MemGet  :: Get(A | S) => S              -- read
12     MemCls  :: TopBot => S                  -- close
13
14  proc memCell :: A | MemCh(A | ) => =        -- memory cell process
15     val | ch => -> hcase ch of
16         MemPut -> do                        -- overwrite existing stored value
17             get nval on ch
18             memCell(nval | ch => )
19         MemGet -> do                        -- send stored value
20             put val on ch
21             memCell(val | ch => )
22         MemCls -> do                        -- close
23             halt ch
24
25  proc memDone :: | Passer( | MemCh([Char] | )) => MemCh([Char] | ), Terminal =
26     | passer => mem, terminal -> do
27     on terminal do                          -- user wants to close
28         hput StringTerminalClose
29         close
30     hcase passer of                        -- check if other proc wants mem
31         Pass -> do
32             fork passer as
33                 pass_mem with mem ->        -- if they do, pass it back
34                 pass_mem |=| mem
35                 neg_passer -> plug
36                 neg_passer, new_passer => ->
37                 neg_passer |=| neg new_passer
38                 => new_passer -> on new_passer do
39                     hput Done                -- but don't request it again
40                     halt
41         Done -> do                          -- close if other proc is already done
42             close passer
43             on mem do
44                 hput MemCls
45                 halt
46
47  -- modified from Example 27:
48  defn
49  proc memAccess :: [Char] | Passer( | MemCh([Char] | )) => MemCh([Char] | ),
50     Terminal =
51  tag | passer => mem, terminal -> do -- terminal is new
52  on mem do
53      hput MemGet
54      get mem_data
55  on terminal do                            -- print mem data, get user input
56      hput StringTerminalPut

```

```

56      put tag ++ " read value: " ++ mem_data
57      hput StringTerminalPut
58      put tag ++ ", please enter a string or press ENTER to close."
59      hput StringTerminalGet
60      get user_input
61      if isEmpty(user_input)                -- check if user wants to close
62      then memDone( | passer => mem, terminal)
63      else do                               -- if they don't, it's the same
64          on mem do
65              hput MemPut
66              put user_input
67          hcase passer of
68              Pass -> do
69                  fork passer as
70                      pass_mem with mem ->
71                      pass_mem |=| mem
72                      neg_passer with terminal -> plug
73                      neg_passer, new_passer => ->
74                      neg_passer |=| neg new_passer
75                      memWait(tag | => new_passer, terminal) -- and keep terminal
76              Done -> do                    -- but other proc might be done
77                  close passer
78                  on mem do
79                      hput MemCls
80                      close
81                  on terminal do
82                      hput StringTerminalPut
83                      put "Other process is done. Press ENTER to close."
84                      hput StringTerminalGet
85                      get _
86                      hput StringTerminalClose
87                      halt
88
89      proc memWait :: [Char] | => Passer( | MemCh([Char]| )), Terminal =
90      tag | => passer, terminal -> do        -- terminal is new
91          hput Pass on passer
92          split passer into mem, neg_passer
93          plug
94          => neg_passer, new_passer ->
95          neg_passer |=| neg new_passer
96          memAccess(tag | new_passer => mem, terminal) -- and used in memAccess
97
98      proc run =
99      | => terminalPing, terminalPong -> plug
100      memCell( "" | mem => )
101      memAccess("Ping" | passer => mem, terminalPing)
102      memWait("Pong" | => passer, terminalPong)

```